

A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges

Meihui Zhong^a, Mingwei Lin^{a,*}, Chao Zhang^{b,*}, Zeshui Xu^c

^a College of Computer and Cyber Security, Fujian Normal University, Fuzhou, 350117, Fujian, China

^b School of Computer and Information Technology, Key Laboratory of Computational Intelligence and Chinese Information Processing of Ministry of Education, Shanxi University, Taiyuan, 030006, Shanxi, China

^c Business School, Sichuan University, Chengdu, 610064, Sichuan, China

ARTICLE INFO

Keywords:

Intrusion detection systems
Graph neural networks
Graph representation learning
Information retrieval
Anomaly detection

ABSTRACT

Intrusion detection systems (IDS) play a crucial role in maintaining network security. With the increasing sophistication of cyber attack methods, traditional detection approaches are encountering more challenges. In recent years, graph neural networks (GNNs) have garnered significant attention in the field of intrusion detection due to their unique ability to capture the relationships within the graph structure of data communications. In this review, we propose a novel taxonomy that categorizes advanced research into three distinct areas: tasks related to graph construction, network design, and GNN models deployment. We detail a generalized design process for GNN-based intrusion detection models, discussing the challenges encountered at each stage. Building upon these discussions, we conduct a systematic survey of existing works. Ultimately, we delve into a thorough exploration of the future research directions and the pending issues in this domain. By adopting a problem-oriented taxonomy and conducting a targeted survey, this review aims to provide scholars with a clear, systematic framework for deepening their understanding and further exploration of the field.

1. Introduction

Intrusion detection is a critical component of maintaining cyber security. Its purpose is to identify threats while preserving the integrity and performance of network deployment. It dynamically detects various attacks from both internal and external sources in real-time, ensuring timely, accurate, and comprehensive detection of intrusions. With the accelerated pace of global digitization, the frequency and complexity of cyber attacks have dramatically increased (Landauer et al., 2023; Javed et al., 2023). These attacks not only compromise the data security of individuals and businesses but can also result in substantial economic losses and harm to reputation (Wenshu Xu, 2023). Given the severity of these threats, there is a pressing need for more sophisticated intrusion detection technologies to counteract and neutralize them.

Deep learning techniques, notably in image recognition and natural language processing, have shown remarkable results (Pouyanfar et al., 2018). In the field of intrusion detection, researchers are exploring machine learning techniques like convolutional neural networks (CNNs) and recurrent neural networks (RNNs). They treat intrusion detection as a feature learning problem (Yan et al., 2024), an approach

illustrated in Fig. 1 (a). They attempt to automatically learn meaningful features from raw communication information. These deep learning models have made significant advancements in domains characterized by large-scale and high-dimensional data, such as network traffic analysis and event correlation in cyber security (Basati and Faghieh, 2022; Kaur et al., 2023).

However, network traffic and system data are highly structured in nature, embodying rich relationships among various elements such as IP addresses, protocols, and host events. Traditional deep learning models struggle to capture these intricate relationships effectively (Cui et al., 2019; Huang et al., 2021; Guo et al., 2023). For intrusion detection, incessant refinements to conventional network structures yield marginal improvements at best. Graph neural networks (GNNs), a subfield of deep learning, leverage the strengths of traditional CNNs within explicit and implicit graph structures (Chen and Wu, 2022). By modeling data like network traffic, communication patterns, and system logs as graphs, GNNs can effectively capture the intricate interactions and relationships embedded within, as shown in Fig. 1(b). Furthermore, GNNs are capable of identifying not only direct relationships between entities but also higher-order connections, like multi-hop context information

* Corresponding authors.

E-mail addresses: mhzhong2021@163.com (M. Zhong), linmwcs@163.com (M. Lin), czhang@sxu.edu.cn (C. Zhang), xuzeshui@263.net (Z. Xu).

Table 1

Summary and comparison of related studies.

Surveys	Year	Types of Analyzed Data	Survey Content
Weller-Fahy et al. (2015)	2015	Network-IDS	Distance and similarity measures
Buczak and Guven (2016)	2016	Network-IDS Host-IDS	Machine learning Data mining Cyber datasets
Umer et al. (2017)	2017	Network-IDS	Machine learning Statistical techniques Flow-based datasets
Gibert et al. (2020)	2019	Malware detection	Machine learning Deep learning
Liu et al. (2018)	2019	Host-IDS with system calls	Data mining Host-datasets Application on systems
Jamalipour and Murali (2022)	2021	Network-IDS	Machine learning Deep learning Reinforcement learning
Nguyen and Reddi (2023)	2021	Network-IDS Host-IDS	Deep reinforcement learning
Bilot et al. (2023)	2022	Network-IDS Host-IDS	Graph neural network Network and host-based datasets
Dong et al. (2023)	2023	IoT sensing environments	Graph neural network IoT sensing datasets
Our Survey	2023	Network-IDS Host-IDS	Graph neural network Graph construction GNN model deployment

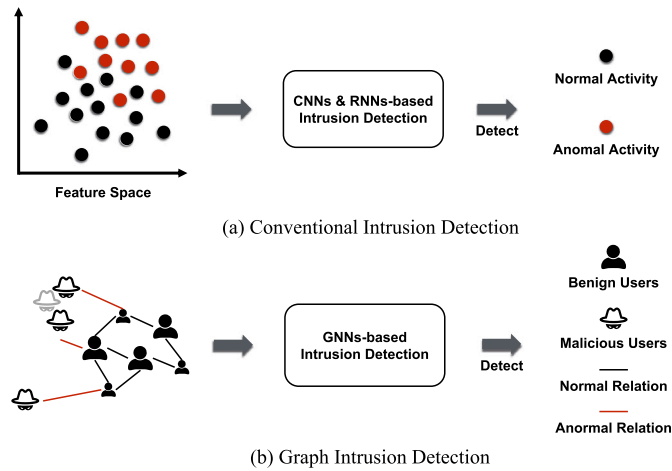


Fig. 1. Toy samples of (a) Conventional Intrusion Detection and (b) Graph Intrusion Detection. Apart from anomalies shown in (b), GNN-based intrusion detection also identifies graph-level anomalies, detailed in Sections 5.2.3. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

(Gao et al., 2023a; Sun et al., 2023). This capability enables GNNs to excel in intrusion detection, especially in identifying complex attack patterns.

The application of GNNs in intrusion detection systems presents distinct challenges. To fully harness the capabilities of GNNs in intrusion detection, several challenges must be addressed. First, constructing an appropriate graph structure from raw data is crucial. Data in the intrusion detection domain, such as network traffic or system data, does not inherently possess a graph structure, necessitating processing to render it compatible with GNNs. Second, designing an effective GNN structure to analyze these graphs and extract meaningful features is challenging. Furthermore, given the efficiency and computational cost requirements of intrusion detection, feature extraction from graph structures incurs extra computational expenses. Hence, the efficient deployment and execution of these models are focal points of research.

In this review, we summarize existing GNN works in intrusion detection, delving into the design process and associated challenges (shown in Table 1). We present a taxonomy that classifies GNN-based intrusion detection into three main categories: graph construction tasks, graph analysis tasks, and GNN model deployment methods. This taxonomy underpins our comprehensive survey of the current literature, offering a structured understanding of the field. We hope that this problem-oriented taxonomy provides readers with a clear, systematic framework, aiding them in gaining a deep understanding of the latest advancements in this domain.

There are some differences between this survey and the existing ones. It is worth mentioning that Bilot et al. (2023) presented the first overview of GNN-based intrusion detection and have made significant contributions in this area. Our work differs from theirs in several key

aspects, the first of which is the taxonomy. They primarily focus on network intrusion detection and host intrusion detection, then categorize based on the type of graph. Their work then showcases various classification levels and different graph neural network models. This classification is distinct from ours, and the two cannot be directly compared side-by-side. Secondly, they overlook the deployment methods of important GNN networks, especially the distributed GNN models, which are of significant interest in the industry. Thirdly, the rapid development of this field means that some recent, significant literature was not included in their previous survey.

While only a few reviews specifically concentrate on GNN applications in intrusion detection, many others address closely related topics. For instance, Weller-Fahy et al. (2015) explored network intrusion detection using similarity metrics, and Liu et al. (2018) reviewed host intrusion detection with a focus on system call graphs. Buczak and Guven (2016) provided an extensive survey on machine learning and data mining methods in this field, emphasizing methodological complexities and challenges. Umer et al. (2017) detailed flow-based intrusion detection, including model architectures and datasets. Gibert et al. (2020) discussed machine learning techniques in malware detection, covering workflows and recent trends. Jamalipour and Murali (2022) and Nguyen and Reddi (2023) examined intrusion detection techniques in IoT, with the former encompassing machine learning, deep learning, and reinforcement learning, and the latter emphasizing reinforcement learning methods. Corona et al. (2013) offered a detailed analysis of adversarial attacks against IDSs, providing insights into vulnerabilities and attack tactics. Dong et al. (2023) concentrated on investigating the advancements of graph neural networks in IoT. The tasks in this field encompass intrusion detection, performance diagnosis, time series forecasting, and environmental monitoring. They found that applying GNNs in IoT intrusion detection is an emerging trend.

To our knowledge, although there are some interesting surveys in both GNN (Zhou et al., 2020; Wu et al., 2021; Zhang et al., 2022b; Yuan et al., 2023) and intrusion detection, the interplay between GNN and intrusion detection data still needs further exploration. Overall, our survey holds significant distinctions from these surveys.

The contributions of this survey are as follows:

- 1) **Comprehensive Review:** We extensively discuss existing literature, thoroughly exploring the interactions between graph neural network and intrusion detection. We present a universal design process, providing insights into each step.
- 2) **New Taxonomy:** We propose a systematic taxonomy for consistently classifying the current and future research on GNN-based intrusion detection methods. We analyze the primary research within each category along with their methodologies.
- 3) **Survey Objective:** We define four research questions (RQs), with the aim of addressing the concerns prevalent in the field through a comprehensive review.
- 4) **Future Directions:** We discuss the limitations of current methods and present several potential future directions.

The remainder of this study is organized as follows: Section 2 encapsulates the insights and findings from the systematic survey on the application of graph neural networks in intrusion detection, aiming to answer specific RQs related to the field, and introduces the research methods of this article. Section 3 introduces the fundamentals of intrusion detection and graph neural networks, emphasizing the motivation behind employing graph neural networks for intrusion detection. Section 4 discusses the challenges associated with applying graph neural networks in intrusion detection. Subsequently, Section 5 categorizes existing methods based on these challenges and presents a detailed discussion centered around four RQs, drawing upon the survey of existing methods. Finally, Section 6 explores the current challenges in this field and proposes potential directions for future research.

2. Survey objective and methodology

2.1. Survey objective

This section encapsulates the insights and findings derived from the systematic survey conducted on the application of graph neural networks in intrusion detection. Specifically, our goal is to answer the following research questions (RQs):

2.1.1. RQ1 (Sections 5.1): how are GNNs applied to intrusion detection?

Our survey indicates that GNNs are predominantly applied for supervised intrusion and anomaly detection within networks. Fewer works delve into exploring the application of GNNs for monitoring system, program, or user behavior within individual hosts.

2.1.2. RQ2 (Sections 5.2 and 5.3): what is the problem definition for intrusion detection based on GNNs?

The problem is typically defined as identifying abnormal traffic and potential security threats present within the network. This process is achieved by classifying unusual targets (such as nodes, edges, and graphs) through learning the inherent graph-structured features within network traffic. Alternatively, the emphasis might shift to detecting abnormal operations or accesses within a host. This is accomplished by training downstream classifiers on representations learned from provenance graphs and system call graphs of system entities. These classifiers are adept at distinguishing between normal and anomalous behavior, providing a robust mechanism for identifying and mitigating potential security risks.

2.1.3. RQ3: what considerations dominate when developing GNN-based intrusion detection?

Researchers focus on several key areas:

- **Graph Construction Methods:** Transforming implicit (inherently non-graph) graph structured data, such as network traffic and system calls, into a graph format suitable for GNN models is a primary concern. This involves determining the representation of nodes and edges, as well as extracting these representations from the raw data.
- **Design and Implementation of Learning Algorithms:** Developing new GNN learning algorithms to more accurately detect network intrusions is a focus area. These algorithms must be capable of handling large-scale graphs and effectively learning complex patterns within the graph structure.
- **Model Performance and Accuracy:** Improving model accuracy and reducing operational costs are crucial metrics for intrusion detection systems. Researchers are working to improve accuracy by enhancing the structure and learning algorithms of GNN models. Furthermore, considering that deployment environments often have limited resources, there is a focus on making the models lightweight, which includes reducing the operational cost of running the models appropriately.
- **Practical Application and Deployment:** Optimizing the real-world deployment of trained GNN models for real-time intrusion detection. This involves computational efficiency and suitability of model for specific cyberspace environments.

2.1.4. RQ4: what are the prevailing trends in IDS based on GNNs?

- **Integration of Other Feature Learning Methods in GNNs:** Integrating other feature learning methods within GNNs. This integration aims to enhance the distinctiveness and robustness of the learned data representations.
- **Research on Dynamic Graph Algorithms in Stream Settings:** Exploring dynamic graph algorithms that operate effectively in stream settings to adapt to changes in real-time. With the advent of dynamic and ever-changing network environments, there is a need for models that can adapt and respond to these changes in real-time.
- **Development of Scalable and Efficient GNN Models:** Designing GNN models that are scalable and efficient. These models are designed to handle large-scale networks and high-dimensional data without compromising high detection accuracy or increasing false positive rates. The literature (Paudel and Eberle, 2020) provides a valuable starting point for researchers aiming to mitigate the challenge of false positives in streaming settings, and reducing the false-positive rate within a secure range would be a significant improvement.

2.2. Survey methodology

In this section, we elaborate on the comprehensive methodology employed for our survey on graph neural networks (GNNs) in intrusion detection systems (IDS), highlighting the systematic process that underpins our research.

Our research commenced with an extensive, systematic search across prominent databases, including Scopus, IEEE Xplore, and the ACM Digital Library. These sources were specifically chosen for their comprehensive collection of scientific papers, which are pivotal in covering the breadth of GNNs applications in IDS. To emphasize the significance of current research and developments in this rapidly evolving field, we focused our survey on publications within a clearly defined time frame. Specifically, our literature review was limited to studies published between January 1, 2019, and February 20, 2024. Our search strategy was multifaceted, involving specific keywords and complex combinations of queries focused on article titles, abstracts, and keywords. We employed a detailed set of queries, such as:

- **For Scopus:** (TITLE-ABS-KEY ("intrusion detection") OR ("anomaly detection")) AND (TITLE-ABS-KEY ("graph neural network") OR (TITLE-ABS-KEY ("graph convolutional network")) OR (TITLE-ABS-KEY ("graph attention network")))) AND PUBYEAR > 2018 AND PUBYEAR < 2025].
- **For IEEE Xplore:** ((("Full Text .AND. Metadata":graph neural network) OR ("Full Text .AND. Metadata":graph convolutional network) OR ("Full Text .AND. Metadata":graph attention network)) AND ((("Full Text .AND. Metadata":intrusion detection) OR ("Full Text .AND. Metadata":anomaly detection))).
- **For ScienceDirect:** "query": Title:(("intrusion detection") OR Abstract:(("intrusion detection")) AND Abstract:(("graph neural network" OR "graph convolutional network") AND Fulltext:(("intrusion detection" AND "graph neural network")), "filter": E-Publication Date: (01/01/2019 TO 02/20/2024).

In our selection process, we adhered to criteria to ensure the inclusion of significant and impactful studies. For journal articles, we imposed a restriction that the journal must have an impact factor of 3.0 or above. For conference papers, we limited our selection to those from conferences ranked C class or higher by the China computer federation (CCF) or those with an h5-index of 3 or above.

Based on the selected survey methodology, a total of 49 articles were chosen for review within the scope of this survey. Fig. 2 provides an intuitive overview of the different categories and subcategories within GNN-based intrusion detection. We have delineated the main categories according to three distinct design phases: data to graph transformation, graph to feature extraction, and GNN models deployment. The subcategories within this framework will be discussed in detail in the corresponding subsections.

Table 2

Key abbreviations used in this article.

Abbreviation	Full Name
RQ	Research Question
GNN	Graph Neural Network
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
IDS	Intrusion Detection System
GRL	Graph Representation Learning
APT	Advanced Persistent Threat
GCN	Graph Convolutional Network
MLP	Multilayer Perceptron
KNN	K-Nearest Neighbor
GAT	Graph Attention Network
GIN	Graph Isomorphism Network
GCNN	Graph Convolutional Neural Network
HGAT	Heterogeneous Graph Attention Network
HetGNN	Heterogeneous Graph Neural Network
TAGCN	Topology Adaptive Graph Convolutional Network
CGNN	Competitive Graph Neural Network
GCNII	Graph Convolutional Network via Initial residual and Identity mapping
GRU	Gate Recurrent Unit
WGAT	Weighted Graph Attention Network
PAGAT	Position Aware weighted Graph Attention Network
UMAP	Uniform Manifold Approximation and Projection
DGCNN	Deep Graph Convolutional Neural Network
PCAP	Packet Capture
ICTG	Interval-Constrained Traffic Graph
VPN	Virtual Private Network
GID	Graph Intrusion Detection
CAN	Controller Area Network
LSTM	Long Short-Term Memory
FCN	Fully Connected Network
NTN	Neural Tensor Network
IoT	Internet of Thing
CPS	Cyber-Physical System
FDI	False Data Injection

In this manuscript, we have introduced a comprehensive taxonomy designed to effectively classify and guide research in the application of GNNs to IDS. This taxonomy is founded on a recognition of the diverse manifestations of network security data and the corresponding need for varied analytical approaches. It categorizes research into two primary types based on the nature of the data: static graphs, which are pivotal for analyzing historical or fixed data structures, and dynamic graphs, essential for real-time monitoring and immediate response to network threats. The taxonomy further distinguishes between node-level, edge-level, and graph-level analyses, each targeting specific facets of network behavior and threat detection with varying granularity. Node-level analysis is tailored to scrutinize individual entities such as hosts or IP addresses, edge-level analysis delves into the relationships between these entities, and graph-level analysis provides a comprehensive view of the network. Additionally, we incorporate provenance graphs and system call graphs into our taxonomy to address unique requirements in host-based IDS, highlighting their significance in tracing data origins, system interactions, and detecting sophisticated threats. For easy reading, we summarize the abbreviations that will be used throughout the article in Table 2.

3. Background

In order to make the presentation of the article more comprehensive, we will introduce the background of intrusion detection and graph neural networks in this section.

3.1. Intrusion detection

The intrusion detection system plays a crucial role in the field of network security (Li et al., 2021b). It continuously monitors and analyzes computer systems and networks to detect signs of unauthorized

access or malicious activities. Its primary goal is to ensure the security, integrity, and availability of digital assets by quickly identifying and responding.

From a data source perspective, intrusion detection systems can be divided into network intrusion detection systems (Network-IDS) and host intrusion detection systems (Host-IDS) (Debar et al., 1999). The former detects potential threats by monitoring network traffic, while the latter focuses on applications and processes running on a single host. Host-IDS can analyze system calls, file activities, and logs more deeply (Pontarelli et al., 2013; Creech and Hu, 2014).

The effectiveness of IDS is often determined by their detection methods. Signature-based detection relies on predefined patterns of known malicious activities. While it is effective against known threats, it may overlook new ones. In contrast, anomaly-based detection establishes a baseline of normal system behavior and flags deviations. It has the potential to identify new threats but might also result in false alarms (Ye et al., 2002; Hwang et al., 2007).

Essentially, its detection process can be defined as two sub-problems: determining whether the current data being detected is normal or anomalous. Such automated systems can typically be implemented using machine learning methods. Modern intrusion detection mechanisms can roughly be divided into three categories: methods based on shallow models (Jain et al., 2022; Gavel et al., 2022; Fraihat et al., 2023), methods based on deep models (Singh et al., 2023; Abdel Wahab, 2022; Wu et al., 2023c), and methods based on ensemble models (Lazzarini et al., 2023; Ren et al., 2023; Louk and Tama, 2023).

To harness vast amounts of data and transform it into valuable knowledge inputs, thereby enhancing the detection capabilities against cyber attacks, researchers continuously integrate machine learning and deep learning techniques to improve network intrusion detection. While convolutional neural networks (CNNs) and recurrent neural networks (RNNs) have gained significant advantages in processing data within flat Euclidean spaces, these models lose their inherent strengths when faced with data presented in high-dimensional manifolds (Han et al., 2019; Zhong et al., 2023). The issue lies in two main aspects. Firstly, the intricate patterns and interactions among data are challenging to describe using sequences or grids, rendering traditional deep learning models incapable of handling such non-Euclidean structures. Secondly, deep neural networks often rely on the availability of vast amounts of costly labeled data (Qi and Luo, 2022; King and Huang, 2023). However, data collected from various devices is often influenced by factors such as network latency and packet loss. This results in the performance of data-driven deep learning models being constrained (Qian et al., 2022).

3.2. Graph neural network

Recently, with the advancements in CNNs and graph representation learning (GRL) (Zhou et al., 2020; Wu et al., 2021), researchers have introduced graph neural networks (GNNs). Unlike traditional neural networks, GNNs are specifically designed to handle graph-structured data, effectively capturing the intricate relationships between nodes and edges. In this section, we first introduce the types of graphs, followed by a brief overview of the design of GNN models.

3.2.1. Graph type

The graph is a fundamental concept in mathematics and computer science used to represent relationships between objects. In graph theory, a graph is typically composed of nodes (or vertices) and edges. A graph G can be defined as an ordered pair $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is a non-empty finite set representing the nodes in the graph, and $\mathcal{E}$ is a finite set representing the edges between nodes. Let $v_i \in \mathcal{V}$ be a node and $e_{i,j} \in \mathcal{E}$ be an edge. The neighbors of the node v_i are represented as $\mathcal{N}_{v_i} = \{v_j \in \mathcal{V} | e_{i,j} \in \mathcal{E}\}$.

In the field of intrusion detection, understanding the type of graphs utilized can enhance the modeling and analysis process. We catego-

size the types of graphs into two: static graphs and dynamic graphs. Static graphs may be more suitable for offline analysis, vulnerability assessment, or historical data analysis. In contrast, dynamic graphs are essential for real-time monitoring and threat detection in continuously evolving network environments.

- **Static Graph.** Static graphs are often used to represent the steady-state structure of a network or host. They are particularly useful for modeling stable relationships and interactions within a network, providing a foundational understanding of its normal behavior. However, static graphs are not adept at capturing temporal dynamics and evolving relationships within a network, which are often crucial for identifying and understanding sophisticated cyber threats.

- **Dynamic Graph.** Dynamic graphs are essential for modeling the temporal aspects of network traffic and system behavior, providing a more comprehensive view of potential security threats. They are particularly relevant for capturing the intricacies of advanced persistent threats (APTs) and other sophisticated cyber attacks that unfold over time (Alshamrani et al., 2019). The analysis of dynamic graphs is computationally more intensive due to their changing structure, requiring advanced modeling techniques and algorithms to effectively extract meaningful insights.

Furthermore, based on whether the edges carry a weight, graphs can be further divided into weighted and unweighted graphs. In weighted graphs, each edge has an associated weight value.

3.2.2. Network design

The core idea behind GNNs is to update the representation of a node by fusing information from its neighbors. In this section, we introduce the general framework of graph neural networks (Xu et al., 2019). Starting with the initial node representation $h^0 = x$, there are two crucial functions implemented at each layer to facilitate the message-passing process:

- **AGGREGATE**, which is responsible for aggregating messages based on the current representation of a node and its neighbors.
- **UPDATE**, which is used to compute the new feature of a node based on its current feature and the incoming messages.

Mathematically, we can define the general framework of graph neural networks as follows:

Initialization: $h^0 = x$. For $l = 1, 2, \dots, L$,

$$\begin{aligned} n_{v_i}^{(l)} &= \text{AGGREGATE}_l \left(\left\{ h_{v_j}^{(l)}, \forall v_j \in \mathcal{N}_{v_i} \right\} \right), \\ h_{v_i}^{(l+1)} &= \text{UPDATE}_l \left(h_{v_i}^{(l)}, n_{v_i}^{(l)} \right), \end{aligned} \quad (1)$$

where $n_{v_i}^{(l)}$ represents the aggregated representation of the neighboring nodes of node v_i , and $h_{v_i}^{(l)}$ represents the representation of node v_i at l th layer.

When we input graph structure and node content information into graph neural networks, the output can be tailored to address different graph analysis tasks by focusing on various mechanisms:

- **Node-Level Task**, which tasks involve making predictions or classifications for individual nodes in the graph. The node-level task primarily utilizes the final node representations after the message-passing process, i.e., $h_{v_i}^{(L)}$ for each node v_i .
- **Edge-Level Task**, which tasks focus on predictions related to the edges or connections between nodes. For this task, GNNs often need to compute representations for edges based on the representations of the connected nodes.

- **Graph-Level Task**, which tasks involve making predictions for the entire graph. This tasks often require a readout function to aggregate information from all nodes and edges in the graph to produce a graph-level representation. Graph-level analysis involves examining the entire graph structure to take into account the current overall environment and identify various types of attacks.

Each of these tasks requires a different design and training approach for the GNN model. For node-level and edge-level tasks, the loss func-

tion is usually defined based on the predictions for individual nodes or edges. In contrast, for graph-level tasks, the loss function is defined based on the predictions for entire graphs. The choice of AGGREGATE and UPDATE functions, as well as the depth of the network (i.e., the number of layers L), also need to be carefully selected based on the specific task at hand.

3.3. Why use GNNs for intrusion detection?

Before delving into the intricacies of recent advancements, it is crucial to grasp the rationale behind the adoption of graph neural networks (GNNs) in intrusion detection. GNNs have changed the detection landscape by revealing latent patterns within data, thereby elevating intrusion detection systems (IDS) performance. The advantages of GNNs in this context can be examined from three perspectives: (1) Structural property of data; (2) Higher-order interaction; (3) Behavioral supervisory signal.

Structural Property of Data: Data, whether sourced from network packets as traffic or from detailed protocol decoding as host events and logs, represents semi-structured information. While these data have no explicitly graph structure, they are inherently interconnected. Traditional detection models often neglect these structural relationships (Duan et al., 2023), but GNNs excel in discerning and learning from them. As a result, GNNs produce high-quality data embeddings, which are crucial for optimal detection performance.

Higher-Order Interaction: GNNs facilitate the flow of information across multi-hop nodes, fostering interactions that extend beyond direct connections through their unique propagation mechanisms (Cui et al., 2019). For instance, once malicious software infiltrates a host system, the entire environment becomes vulnerable to subsequent attacks. Nodes that exhibit similar patterns of information propagation are likely to be compromised, an effect known as collaborative filtering. Standard neural networks, such as CNNs and RNNs, cannot handle the connections between input data nor capture the potential distribution structure of the data. This is because they process node features as isolated inputs without considering the structural similarities between data. Their primary limitation is their focus on first-order interactions and isolated data analysis. High-order interactions represent a critical pattern for the transmission of information in intrusion detection. The emergence of GNNs allows models to capture these hidden patterns more effectively.

Behavioral Supervisory Signal: The scarcity of attack instances during data collection poses a challenge for detection models. GNNs mitigate this by leveraging behavioral supervision signals within the graph structure during the learning process. As information propagates, the behavior of a labeled node can influence other unlabeled data within its scope (Liu et al., 2023a). Consider the example of computer worms: the attack behavior made by a certain attacker IP is quite sparse compared to its normal behavior. If detection models focus solely on these infrequent attack behaviors, they may suffer from a high false negative rate. However, GNN-based models can effectively improve by incorporating additional context (Yu et al., 2023), such as the IP's non-target behavior of repeatedly scanning other IPs within a certain timeframe. These models can significantly enhance the performance of intrusion detection systems in predicting attack behaviors.

4. Challenges of applying GNNs to intrusion detection

We propose a novel problem-oriented taxonomy to categorize existing GNN-based models and provide an in-depth discussion about them. Specifically, to effectively utilize GNNs for intrusion detection, there are several crucial challenges that researchers need to address, which are as follows:

- 1) How can cyber security data be effectively adapted to a graph structure?

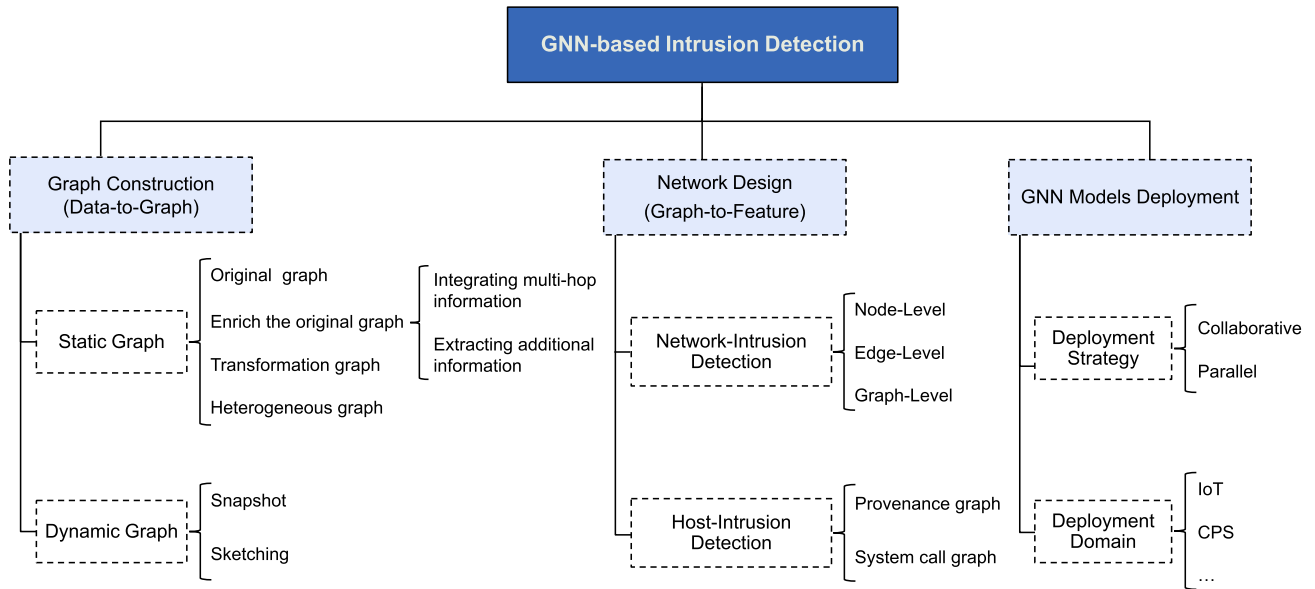


Fig. 2. A taxonomy of GNN-based intrusion detection.

- 2) What features should be learned from the graph structure?
- 3) How should GNN models be deployed?

In the following, we will elaborate on these three challenges one-by-one.

4.1. Graph construction

Before diving into the intricacies of graph-based intrusion detection, it is essential to understand how data, especially in the realm of cyber security, can be transformed into graph structures. This transformation is crucial as it allows for the application of graph-based algorithms and methodologies to datasets that were traditionally not viewed in a graph context. While network traffic and host data are inherently structured, they mostly exist as implicit graph structures. We focus on how to transform conventional network data into a graph structure. This encompasses choosing appropriate representations for nodes and edges, as well as effectively capturing relationships and patterns within the network.

A well-designed graph structure can reveal more high-quality information about the data. For intrusion detection systems, input data requires meticulous design to form informative graph structures. For instance, nodes represent hosts, and edges represent relationships. The solutions go beyond these, with the key focus on constructing graphs that capture complex interactions between data, such as network traffic or host log information.

4.2. Network design

A well-designed GNN model can learn higher-quality information, leading to superior performance in subsequent detection tasks. Existing studies focus on network structure design, including propagation and aggregation methods, each with their own advantages and disadvantages. Regarding detection tasks, the emphasis lies in designing GNN structures to learn higher quality graph representations while maintaining a balance between model complexity and detection performance.

GNNs have a myriad of hyperparameters, such as the count of neurons in each network layer and the count of neighboring nodes. Standard GNN architectures are designed to learn features under ideal conditions, where the observed graph data (structure, attributes, labels) is sufficiently complete to provide ample information. However, in network or host environments, there is a high likelihood of packet loss,

IP address spoofing, or insufficient labeled data. The question of how to derive a strong model from weak information represents a pivotal research challenge in model architecture design (Liu et al., 2023a).

4.3. GNN model deployment

While GNNs can extract richer structural features distinct from those obtained by CNNs or traditional machine learning methods, they come at a significantly higher computational cost (Cai et al., 2018; Cui et al., 2019; Ma et al., 2023). Therefore, the final concern relates to the computational costs of GNNs, given the real-time nature of intrusion detection. The computational operations introduced by GNNs add to the complexity of the model, making efficient model deployment a crucial research task for intrusion detection.

Based on these challenges, existing research has been categorized into tasks related to graph construction, graph analysis, and the deployment of GNN models. Next, we provide a detailed presentation of these classifications and analyze how these studies address the aforementioned issues.

5. Existing research

Recent advancements in graph-based methodologies have underscored their efficacy in diverse domains, notably within the realm of intrusion detection, as delineated in Section 3.3. However, there are numerous challenges that arise during the design process of these methods, outlined in Section 4. As such, we have proposed a problem-oriented taxonomy that systematically organizes the body of existing research, as depicted in Fig. 2.

5.1. Data-to-graph

Graph structures excel at modeling network activities or host events, as they intuitively illustrate the structural/relational information between different entities or events (Ma et al., 2023). To represent traffic or log data with graphs, it is imperative first to extract relevant data from the dataset and transform it into graph components based on the entities involved. Once the raw data is stored in the graph, various tools and techniques can be utilized to analyze the graph data. This approach potentially offers a powerful alternative to traditional intrusion detection, providing deep insights into the behaviors among entities and identifying anomalies (Messai and Seba, 2023).

The conventional method of graph construction involves using IP addresses as nodes in the graph, with data packets serving as the edges between nodes. In addition to the conventional method of graph construction involving IP addresses and data packets, we recognize the potential of other network data types in graph-based IDS. NetFlow data, for instance, can be effectively represented as a graph where nodes represent network elements such as IP addresses or devices, and edges signify the flow of traffic between them. This representation is instrumental in analyzing network traffic patterns and detecting anomalies. Similarly, PCAP network packet capture data offers a rich source for graph construction. In graphs derived from PCAP data, nodes can be structured to represent endpoints like IP addresses or ports, and edges indicate packet transmissions. This detailed representation allows for a more granular analysis of network interactions.

Log files from various network devices and applications are also amenable to graph representation. In such graphs, nodes could represent entities like users, devices, or applications, while edges could denote interactions or events logged, facilitating the detection of unusual patterns of behavior or security breaches.

Furthermore, we acknowledge the utility of transforming other types of network data, such as DNS queries and HTTP requests, into graph structures. In the graph structure of HTTP request data, nodes can represent users, web pages, or servers. Edges denote HTTP requests and responses, indicating the interaction between a user and a specific web page or server.

Several issues arise from this: (1) GNNs need to learn edge embeddings to classify data packets, but does learning node embeddings not matter? (2) Considering that exploring different types of embeddings or incorporating additional node and edge attributes might enhance the graph's representational power, should GNNs learn other hidden information within the graph? (3) Considering computational efficiency and model performance, should operations be performed on the entire graph, or should sampling be conducted on representative subgraphs?

With these questions in mind, we delved deeply into the various existing methods for constructing graph structures. The various methods are summarized in Table 3, and we have categorized them into two main types: static graphs (Section 5.1.1) and dynamic graphs (Section 5.1.2).

5.1.1. Static graph

Most studies (Wang et al., 2020; Protogerou et al., 2021; Yehezkel and Elyashiv, 2022; Govindaraju et al., 2022; Lo et al., 2022; Lan et al., 2022; Caville et al., 2022; Wang et al., 2023; Nguyen and Kashef, 2023; Gao et al., 2023b; Ko et al., 2023; Biswas et al., 2023; Agiollo et al., 2023; Hu et al., 2023; Wu et al., 2023d; Wang et al., 2024; Altaf et al., 2023b; Reka et al., 2024) utilized the original graph construction methods. For Network-IDS, this typically involves representing IP addresses as nodes and the interactions or connections represented by data packets as edges. For Host-IDS, it involves explicit graph structures, such as provenance graphs and system call graphs. Applying GNNs on these original graphs presents two main issues: compatibility and effectiveness. The fixed patterns of original graph structures often limit analysis to edge classification, where current GNN models underperform due to inadequate utilization of edge features (Gong and Cheng, 2019).

One strategy to address the first issue is to highlight different information through graph transformation. Some studies have transformed graph structures into line graphs, which allow for the simultaneous use of node and edge features and convert edge classification tasks into node classification tasks.

In graph theory, a line graph or covering graph is a derivative graph constructed from an original graph wherein each vertex of the line graph corresponds to an edge of the original graph. Adjacency between two vertices in the line graph is established if and only if their corresponding edges in the original graph share a common vertex. Converting to a line graph facilitates mutual information flow between traffic from the same source node.

Chang and Branco (2021) constructed a bipartite graph with separate sets of source and target nodes and then transformed it into a corresponding line graph. This transformation addresses the issue of disjoint traffic from source nodes in the bipartite graph, which hinders direct information propagation. Differing from the model proposed by Chang and Branco (2021), GBA-IDS (Zhu and Lu, 2022) implemented a linear variant of GraphSAGE, called Line-GraphSAGE, to learn edge embeddings within their line graph.

However, the transformation into line graphs may incur significant computational costs and may not adapt well to all graph types (e.g., heterogeneous graphs), potentially generating redundant edges. Efforts have been made to improve and address this issue. Friji et al. (2023) proposed a novel graph construction method that models flows as nodes, linking flows with common IP origins or destinations. Deng et al. (2023) formulated three rules to construct the interval-constrained traffic graph (ICTG), considering three types of traffic transmission and time intervals to establish connections. The first rule pertains to traffic flows F_i and F_j sharing a common source IP; the second involves flows where the destination IP of F_i matches the source IP of F_j ; the third rule applies a temporal constraint, establishing links only between flows F_i and F_j with a time interval less than a predefined threshold T .

These graph construction methodologies offer the advantage of not relying directly on IP addresses or ports. Since attackers frequently utilize virtual private networks (VPNs) to obscure their true IP addresses, this strategy somewhat mitigates this issue. When the detection of attacks or anomalies is conceptualized as a node classification task, it essentially involves classifying all traffic emanating from a source IP. Consequently, once malicious traffic is identified, its source node is labeled as malicious, and all outgoing traffic from it is deemed to be of a malicious nature. Traffic forwarding systems, such as switches, would then discard it.

In terms of the second issue, one strategy is to enrich the original graph structure by adding edges and hyperedges, such as links between two-hop neighbors. For instance, Zheng and Li (2019) proposed a graph construction method (GCN-TC) different from the original one. To accommodate GCN models based on node classification, GCN-TC represented network flows as nodes in the graph and adds edges between flows with common sending or receiving nodes. Furthermore, they introduced a K-hop adjacency model, allowing the model to integrate information from neighbors beyond the one-hop neighbors. Hu et al. (2021) proposed a multi-hop graph embedding technique that considered the multi-hop connectivity between nodes to construct the graph. This enabled the model to detect botnet components that propagate commands and control information through multiple levels.

Beyond integrating information from multi-hop neighbors, another strategy is to extract additional information from the original features to serve as attributes for nodes and edges. Alert-GCN (Cheng et al., 2021) utilized one-hot encoding and similarity measures for graph construction. Rather than determining whether to add edges between nodes based on original data packets, Pujol-Perich et al. (2022) proposed using the degree of similarity as the criterion. In their model, each node represents a network flow, and each edge signifies the similarity between two flows, with cosine similarity employed as the edge weight. A threshold, determined through cross-validation, is applied to decide whether to establish an edge, thereby refining the graph structure.

In addition to classifying information within data manually based on expert knowledge, an increasing number of studies are exploring the integration of machine learning and deep learning methods. These methods were used to extract additional information that can be added to the attributes of nodes or edges. For instance, Zhang et al. (2023b) introduced a continuously iterative network reconstruction technique in their graph intrusion detection (GID) framework. They simultaneously considered the construction of the graph and the parameter learning of the GNN model as part of the model training. This approach not only facilitated the creation of a graph structure that closely aligns with the true data distribution (based on some prior information) but also

Table 3

Summary of GNN-based intrusion detection.

Analyzed Data Types	Graph Types	Graph Construction Methods	Network Design	Graph Embedding Methods	Method	Year
Network-IDS	Static Graph	Original graph	Statistical Analysis	Graph-level	BotMark Wang et al. (2020)	2020
			Edge/Node-MLP	Node-level	GraphNET Protogerou et al. (2021)	2021
			Graph2Vec	Node-level	Yehezkel and Elyashiv (2022)	2022
			GCN	Node-level	IDF-AGNN-HYB-WMA-CSOA-IoT Govindaraju et al. (2022)	2022
			GraphSAGE	Edge-level	E-GraphSAGE Lo et al. (2022)	2022
			E-GraphSAGE	Edge-level	E-minBatch GraphSAGE Lan et al. (2022)	2022
			E-GraphSAGE	Edge-level	Anomal-E Caville et al. (2022)	2022
			GAT + LSTM	Node-level	N-STGAT Wang et al. (2023)	2023
			GAT	Node/Edge-level	TS-IDS Nguyen and Kashef (2023)	2023
			GCN	Node-level	Gao et al. (2023b)	2023
			GAT	Node-level	Ko et al. (2023)	2023
			GCN	Node-level	Biswas et al. (2023)	2023
			GIN	Node-level	GNN4IFA Agiollo et al. (2023)	2023
			Graph2Vec	Node-level	Graph2vec+RF Hu et al. (2023)	2023
			GAT	Node-level	Wu et al. (2023d)	2023
			Spatial-Temporal GCN	Node-level	APS-T Wang et al. (2024)	2023
			GCN + GAT	Node-level	Altat et al. (2023b)	2023
			GCNN	Node-level	COCG-MSA-GCNN-MA-ID-MANET Reka et al. (2024)	2024
	Transformation graph	Enrich the original graph by integrating multi-hop information	E-GraphSAGE + GAT	Node-level	Chang and Branco (2021)	2021
			GraphSAGE	Edge-level	GBA-IDS Zhu and Lu (2022)	2022
			TAGCN + Attention Mechanism	Node-level	FT-GCN Deng et al. (2023)	2023
			GCN + GAT	Node-level	Friji et al. (2023)	2023
		Enrich the original graph by extracting additional information	GCN	Node-level	GCN-TC Zheng and Li (2019)	2019
			Distance-based Graph Embedding	Node-level	Hu et al. (2021)	2021
			GCN	Node-level	Alert-GCN Cheng et al. (2021)	2021
		Heterogeneous graph	GCN	Node-level	Pujol-Perich et al. (2022)	2022
			GCN	Node-level	GID Zhang et al. (2023b)	2023
			HGAT	Node/Edge-level	Ao et al. (2021)	2021
Host-IDS	Dynamic Graph	Snapshot	CGNN	Edge-level	HetGLM Sun and Yang (2022)	2022
			GCN	Edge-level	NE-GConv Altat et al. (2023a)	2023
			GCN + Attention Mechanism	Edge-level	AddGraph Zheng et al. (2019)	2019
			GCNII + GRU	Edge-level	DLGNN Duan et al. (2023)	2023
		Sketching	-	Edge-level	Euler King and Huang (2023)	2023
			-	Node-level	Messai and Seba (2023)	2023
			Node2Vec	Graph-level	SNAPSKETCH Paudel and Eberle (2020)	2020
			PAGAT + WGAT	Graph-level	GLAD-PAW Wan et al. (2021)	2021
			GAT	Node-level	GLD-Net Guo et al. (2022)	2022
			Manifold Learning (UMAP)	Node-level	DMTR Zhong et al. (2023)	2023
	Static Graph	Provenance Graph	Graph Embeddings	Node-level	Zerhoubi et al. (2020)	2020
			Word2Vec + GCN	Graph-level	DeepHunter Wei et al. (2021)	2021
			GraphSAGE	Node-level	THREATTRACE Wang et al. (2022)	2022
			HetGNN	Graph-level	OC-DHetGNN Huang et al. (2022)	2022
			Statistical Analysis (Similarity Calculation)	Node-level	Paradise Wu et al. (2023e)	2023
	System Call Graph	System Call Graph	Graph Transformer (GT)	Graph-level	LogGD Xie et al. (2022)	2022
			GCN + GraphSAGE + GIN	Node-level	Lakha et al. (2022)	2022
Controller Area Network-IDS	Dynamic Graph	Snapshot	Statistical Analysis + LSTM-RNN	Graph-level	Jedh et al. (2021)	2021
			GAT	Graph-level	CAN-GAT Xiao et al. (2023)	2023
		Sketching	Statistical Analysis	Graph-level	Islam et al. (2022)	2022
			DGCNN	Graph-level	Zhang et al. (2023a)	2023

* The methods presented in King and Huang (2023); Messai and Seba (2023) offer a framework designed to accommodate the dynamic variations in data, allowing for a variety of encoder models to be employed, such as GCN, GAT and GraphSAGE.

refined and updated the graph structure through further learning from graph embeddings. Specifically, GID employs a multi-head weighted cosine similarity network learning method to obtain the network structure A of the input data X . The similarity is calculated as follows:

$$s_{ij}^k = \cos(b_k^l \odot v_i^l, b_k^l \odot v_j^l), \quad (2)$$

$$s_{ij} = \frac{1}{m} \sum_{k=1}^m s_{ij}^k, \quad (3)$$

where $\odot$ represents element-wise multiplication, and the average similarity s_{ij} is the mean of the similarities calculated by the m heads of weight vectors. In Eq. (3), v_i^l and v_j^l represent the vectors of the nodes i and j at the l -th node embedding level, respectively. b_k represents

the k -th weight vector (the k -th head) that will be updated during the training process. If $k = 1$, then v^1 is the original features of the nodes.

To make the network structure closer to the real data distribution, the authors also used regularization techniques to design the network structure's loss function. This function aims to make different types of data sparsely connected or even not connected. The authors designed the network regularization structure loss L_N as Eq. (4) by improving the Dirichlet energy function $\Omega(\mathbf{A}, \mathbf{X})$ and the $l1$ -norm $f(\mathbf{A})$.

$$\begin{aligned} L_N &= \alpha \Omega(\mathbf{A}, \mathbf{X}) + f(\mathbf{A}) \\ &= \alpha \Omega(\mathbf{A}, \mathbf{X}) - \frac{\beta}{n} \mathbf{1}^T \log(\mathbf{A}\mathbf{1}) + \frac{\gamma}{n^2} \|\mathbf{A}\|_F^2, \end{aligned} \quad (4)$$

where α, β, γ are non-negative hyperparameters, and $\mathbf{1}$ represents an all-ones matrix. This equation represents the overall loss, which includes the sum of network smoothness, connectivity, and sparsity losses. The first term is the Dirichlet energy, which ensures the smoothness of the network. The second term applies the logarithmic barrier to the vector $\mathbf{A}\mathbf{1}$ to ensure the positivity of the degrees and the connectivity of the network. The last term calculates the Frobenius norm of the adjacency matrix in the network to control its sparsity.

However, systems based on homophily and structural equivalence (Bowman et al., 2020), which are constrained by homogeneous graph models, cannot fully exploit the network or host environment. In real-world scenarios, heterogeneous graphs, which encompass a variety of node and edge types, offer a more robust framework for intrusion detection, particularly when operating under real-time constraints with limited data.

Ao et al. (2021) established a heterogeneous graph named IP-Domain-Graph to capture the relationships between IP addresses and the domains to which they send malicious requests, differing from the conventional homogeneous graph construction methods. In homogeneous graphs, all nodes are of the same type, whereas heterogeneous graphs contain multiple types of nodes and edges, encompassing more information about network behaviors and attacker patterns. In the IP-Domain-Graph, nodes represent IP addresses and domains, and edges indicate the behavior of IP addresses sending malicious requests to domains. This approach described the interaction between IP addresses and their target domains, as well as the characteristics of attackers' behaviors. Sun and Yang (2022) constructed a heterogeneous graph utilizing diverse types of log files, encapsulating a range of network entities such as users, devices, and processes. They specify meta-paths as edges, to represent complex relationships among these entities. These meta-paths consist of four core internal network elements and six relationship types.

Existing methods generally deal with either traffic data or host data in isolation. NE-GConv (Altaf et al., 2023a) integrated both host and flow features within IDS, specifying a graph structure capable of handling both node and edge features simultaneously. In this model, IoT sessions are represented as nodes using a combination of IP addresses and port numbers. The features of these nodes are host-specific attributes like payload fields. Inter-entity communications are represented as edges, with selected flow field attributes such as duration, packet count, and arrival time serving as edge features.

5.1.2. Dynamic graph

Dynamic graphs, which evolve over time, are crucial for real-time intrusion detection. Static graphs are insufficient for streaming settings where data and host logs change dynamically. Most graph-based network traffic detection tools aim to analyze past traffic to understand previous attacks. Researchers are contemplating how to make graphs reflect real-time changes as the data evolves.

One strategy is to generate snapshots of the dynamic environment, that is constructing new graphs within a certain time frame, allowing the graphs to approximate and describe the full picture of the data. AddGraph (Zheng et al., 2019) constructed dynamic graphs that change

over time. To address the issue of imbalanced anomaly data, AddGraph generated anomaly samples based on each edge, treating them as anomaly edges. CAN-IDS (Jedh et al., 2021) constructed dynamic message-sequence graphs, transforming message sequences within specified time intervals into directed graphs. In these message-sequence graphs, node labels represent CAN IDs, while edge labels represent the number of message transmissions between CAN IDs. Furthermore, these graphs change along with the timestamp t . Considering that the number of nodes can lead to exponential growth in graph computations, Xiao et al. (2023) treated each timestamp as a node and constructed graph G by selecting a fixed set of 50 timestamps at each instance.

Duan et al. (2023) captured the contextual evolution of communications between IP pairs through continuous snapshots, converting network traffic into a series of spatiotemporal graphs. King and Huang (2023) viewed interactions occurring within discrete units of time in the network as a series of snapshots, represented by graph G_t . In this representation, V is the set of entities in the network that have interactions, and $E_t = \{(u, v) \in V\}$ represents interactions occurring within the time interval t .

While some works have considered real-time graph representations in networks, these are typically used for monitoring snapshots of the entire network, which can be resource-intensive. In the context of flow graphs, sketching is another technique for dynamic graph representation, projecting high-dimensional data into low-dimensional feature vectors.

Messai and Seba (2023) introduced a graph-based framework that utilizes graphs to depict activity windows in network traffic, subsequently classifying the traffic as normal or anomalous. Similar to other approaches, this framework operates in two phases: initially generating graph structures based on all available attributes in traffic communication, followed by training GNNs to learn graph embeddings. Uniquely, each edge in the graph carries a timestamp, recording dynamic network communications. This allows for the dynamic addition and removal of vertices and edges as communications evolve over time.

Paudel and Eberle (2020) proposed an unsupervised, general-purpose graph sketching technique for intrusion detection within streaming setting environments. This technique conceptualizes network entities—such as DNS servers, web servers, workstations, and devices with internet connectivity—as nodes within a flow graph. The interactions between these entities are depicted as edges. In the context of dynamic graphs, traffic for each continuous fixed time period is represented as an individual graph.

The controller area network (CAN) serial communication protocol bus, integral to real-time applications, necessitates robust real-time intrusion detection systems. Existing real-time detection mechanisms are either limited to identifying a narrow spectrum of attack types or require an extensive volume of CAN messages during the detection process, thereby complicating the implementation of real-time functionalities. Islam et al. (2022) segmented CAN bus messages into multiple windows, constructing graphs for the messages within each window, and then attempted to discern the connections between each window. Zhang et al. (2023a) developed a CAN bus detection system capable of detecting all known attacks in as short as 3 milliseconds. This system involves constructing directed graphs of CAN messages over specified time intervals, with node features representing data content in CAN messages and edge features representing the frequency of typical CAN ID pairs.

Guo et al. (2022) argued that such models might overlook changes in data topology connections. To address this, they proposed GLD-Net, a model adept at learning both flow and topology features from time-series stream data. GLD-Net initially generates dynamic DDoS topology graphs from time series, a process divided into two phases: the initial phase converts traffic data into node or edge features, with traffic within certain time intervals forming distinct [IP-Feature] pairs stored in a feature table. The subsequent phase involves the construction of connection graphs, adding feature attributes to nodes or edges based on

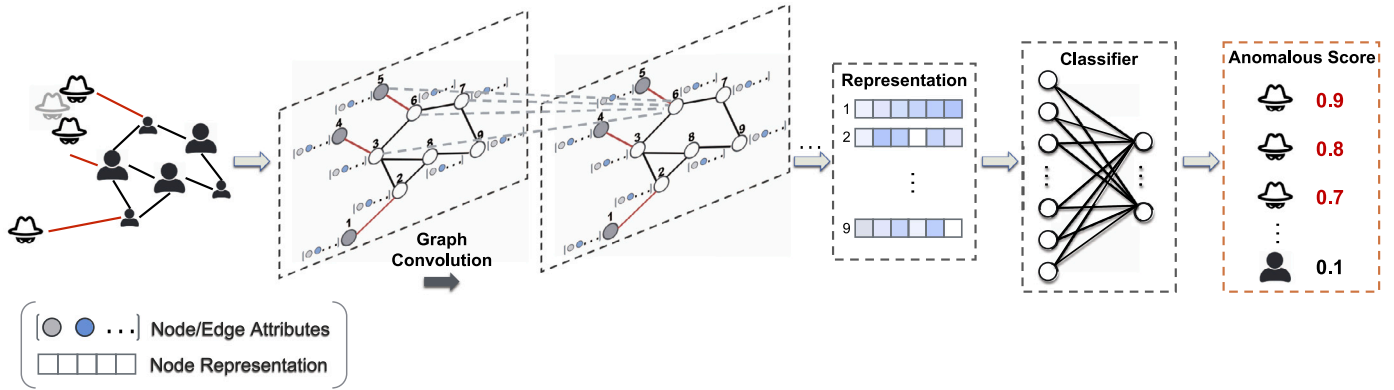


Fig. 3. A toy example of graph convolution for node or edge-level intrusion detection (static graph). The graph convolutional layer is employed to learn representations for nodes or edges. Classifiers such as fully connected networks (FCN) or multi-layer perceptrons (MLP) are trained to assign anomaly scores. The top-k data points are delineated as anomalies.

the feature table, and deciding whether to add these attributes based on whether two nodes have communication (flows) between them. This results in topological feature maps G on m sub-time intervals ($G = G_1 \cup G_2 \cup \dots \cup G_m$). In GLD-Net, feature attributes include not only statistical data but also the final graph's connection status, packet tags, and centrality features.

Given resource constraints in streaming settings, there are insufficient resources for graph transformations, particularly when compared to systems based on static graphs. Dynamic graph constructions often incorporate additional information to enrich the graph, enabling models to learn data dynamics within the constraints of time and resources.

5.2. Graph-to-feature in network intrusion detection

Given a well-constructed graph, a challenge lies in defining the information propagation and fusion strategies within the GNNs (Xiao et al., 2021). The overall workflow for extracting node-based and edge-based representation from a graph is shown in Fig. 3.

With node content, edge attributes and graph structures serving as inputs for GNNs, their outputs can be tailored to accomplish various graph analysis tasks through one of the following three mechanisms:

- **Node Level:** Outputs at the node-level focus on the attributes or features of individual nodes, typically employed for tasks such as node classification, regression, or label prediction, where the output for each node is independent.
- **Edge Level:** Edge-level outputs concentrate on the attributes or relationships of edges within the graph, suitable for predicting or classifying connections between nodes, as in link prediction or edge classification tasks.
- **Graph Level:** Outputs at the graph-level are concerned with the attributes or features of the entire graph. This mechanism is utilized for classification or regression tasks related to the whole graph, such as graph classification, graph regression, or graph similarity assessment tasks.

5.2.1. Node-level

GraphSAGE (Hamilton et al., 2017), as an early extension of GCN (Kipf and Welling, 2017), was the first method designed for information propagation within large graphs. This was a significant advancement for network intrusion detection systems (NIDS), especially considering that the graph structures derived from traffic data are typically extensive. However, the primary objective of NIDS is to identify anomalous network traffic, framing the issue predominantly as an edge classification problem, since that traffic information is integrated into the edge attributes of the graph. This makes it challenging for GCN and GraphSAGE, which are used for node and graph-level tasks, to function effectively.

As described in Section 5.1, many studies start with graph construction methods, transforming the problem into a node task while also utilizing edge information. GCN-TC (Zheng and Li, 2019) transformed the graph and subsequently used GCN for node classification.

Chang and Branco (2021) converted the original edge classification task into a node classification task by constructing a line graph. They introduced enhanced E-GraphSAGE and E-ResGAT algorithms. The introduced E-GraphSAGE learns extensive neighborhood information by sampling nodes within K hops. The E-ResGAT method integrates residual learning into the GNN, enabling the leveraging of multi-layer information within the graph. To consider the entire neighborhood collectively, the graph attention mechanism is also incorporated into the method.

Guo et al. (2022) employed graph attention networks (GAT) to learn fused features of the topology graph, which include both topological structural features and flow features. The output is a spatial sequence over the domain. Subsequent to the GAT, a long short-term memory (LSTM) network is utilized to mine relationships within the sequence. A fully connected layer and softmax function are then applied for classification.

Deng et al. (2023) introduced a network that concurrently learns both topology and statistical information. Building upon the foundation of the TAGCN network (Du et al., 2017), it incorporates a node-level spatial attention mechanism, aiming to extract features of key nodes.

5.2.2. Edge-level

Addressing edge classification challenges necessitates not only improved graph construction methods but also enhancements to network models to fully exploit edge information. The objective is to ensure that the model's final output is edge embeddings, rather than node embeddings, to better capture the nuances of network interactions.

Lo et al. (2022) observed that the message-passing functions in traditional GraphSAGE algorithms rarely take into account features other than node features, which is insufficient for improving detection methods based on these models. In response, they introduced E-GraphSAGE for network intrusion detection in IoT networks. The distinction between E-GraphSAGE and the original GraphSAGE is that the former incorporates edge features into its inputs for edge feature fusion. E-GraphSAGE employs a specific node aggregation function and allows capturing both the edge features of graphs as well as the topological information.

GCN, with its simple, scalable architecture and computational efficiency, has become one of the most commonly used methods. AddGraph (Zheng et al., 2019) leverages both GCN and attention mechanisms to effectively learn information from dynamic graphs under a sequential model, as shown in Fig. 4.

Furthermore, E-minBatch (Lan et al., 2022) pre-samples nodes to obtain graph structures, allowing E-minBatch GraphSAGE to adapt to

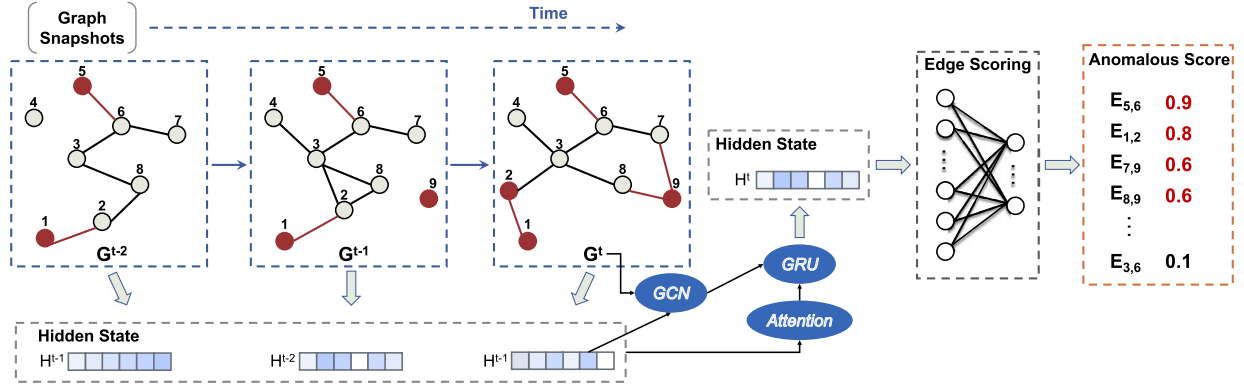


Fig. 4. A toy example of graph convolution for node or edge-level intrusion detection (dynamic graph). For each graph snapshot, the GCN is utilized to derive node embeddings from the temporal graph at discrete time intervals. Gated recurrent unit (GRU) synthesizes the current hidden state by integrating these node embeddings with preceding hidden states. The edge scoring function such as FCN is trained to allocate anomalous scores. The top-k data points are delineated as anomalies.

complex network structures. Compared to the original E-GraphSAGE model, E-minBatch can handle large-scale data inputs more effectively.

Caville et al. (2022) noted that most GNN-based intrusion detection methods rely heavily on labeled traffic, which limits the scale and structure of input data. These methods also depend on the use of node features and neglect important edge information, leading to decreased detection accuracy. They addressed this by randomly shuffling the edges in real networks to create corrupted network graphs. This is done by shuffling the edge feature matrix in row X . Anomal-E then uses edge information and learns edge embeddings in a self-supervised manner, capturing key traffic features. The entire process uses E-GraphSAGE to extract both the edge embeddings and the corrupted edge embeddings.

HetGLM (Sun and Yang, 2022) provides an anomaly edge detection algorithm based on heterogeneous graphs. HetGLM fundamentally utilizes the CGNN (Zhang et al., 2022a) model framework, consisting of an encoder and a decoder, which together differentiate between anomalous and normal edges. Within the encoder, both fully connected network (FCN) and GNN encoder are employed to learn the relationships between nodes. The FCN is used to learn the attribute features of edges, while the GNN is utilized to generate embeddings for user and device nodes. Distinct from CGNN, HetGLM leverages the diversity of heterogeneous graphs by employing a meta-path-based random walk strategy for neighbor node sampling. This approach, coupled with the use of attention mechanisms during the aggregation of node attributes, aids in capturing the variations between different meta-path samplings. The aggregation operation is denoted as:

$$w^{p_j} = \frac{1}{|V_A|} \sum_{v \in V_A} \alpha_A^T \cdot \tanh(W_A \cdot h_v^{p_j} + b_A),$$

$$h_v = \sum_{p \in P_A} \frac{\exp(w^{p_j})}{\sum_{j \in |P_A|} \exp(w^{p_j})} \cdot h_v^{p_j},$$
(5)

where w represents the aggregated representation and V_A represents the set of nodes in the type A . α_A , W_A , b_A are the weight matrices, $h_v^{p_j}$ represents the node embedding vector obtained through the meta-path P_j , and P_A is the set of meta-paths.

NE-GConv (Altaf et al., 2023a) is an enhancement of the GCN model. This model has two layers of GCN, used to learn host and network features in the graph structure separately. Ultimately, the edges are classified as either normal or malicious. DLGNN (Duan et al., 2023) employs dynamic GNNs to extract spatial information from each discrete snapshot, capturing the contextual evolution of communications between IP pairs through consecutive snapshots. Specifically, DLGNN integrates GCNII (Chen et al., 2020) layers to extract spatial information within line graph structures, where GCNII, compared to the vanilla GCN, mit-

igates overfitting. Furthermore, it incorporates GCNII-GRU layers to capture the temporal information of discrete snapshots.

5.2.3. Graph-level

In general, relatively scarce in the literature involves graph-level algorithms for network intrusion detection. This scarcity can be attributed to the nature of intrusion detection tasks, which often focus on identifying anomalous activities at the node or edge levels, such as unusual traffic patterns or suspicious connections between nodes. Graph-level analysis, on the other hand, would entail examining the entire network graph for signs of anomalous behavior, which is a more complex and computationally intensive task.

BotMark (Wang et al., 2020) takes into account both graph-based traffic behavior and statistically-based traffic behavior, as shown in Fig. 5. To identify Bot anomalies, they perform a hybrid analysis of these behavioral features, including anomaly analysis of the ego-net (a subgraph centered around a single node), stability analysis of the data flow, and similarity analysis. Rather than constructing a visual graph structure, they directly extract features based on the graph.

SNAPSKETCH (Paudel and Eberle, 2020) is an unsupervised graph representation technique designed for stream graphs, and the method process is shown in Fig. 6. This technique utilizes the node2vec (Grover and Leskovec, 2016) to generate snapshots of data streams at any given time t , embedding high-dimensional graph-structured data into low-dimensional vectors. It then employs Simplified Hashing for sampling to retain low-storage, low-dimensional sketch vectors. The main design principle of SNAPSKETCH is that the graph structure of anomalous graphs undergoes a sudden change rather than a slow evolution. The algorithm is adept at detecting such anomalies, proving helpful in identifying network attacks within computer networks, such as port scanning and denial-of-service attacks.

In the GLAD-PAW (Wan et al., 2021), it is recognized that the same communication event might occur at different positions within a session. However, after basic graph construction, this positional information is lost. Therefore, the model keeps track of the positions where each communication event occurs and incorporates position information layer (PAGAT) with learnable embeddings. In the model, the position-aware node embeddings are denoted as:

$$h_{s,i} = h_{v_{s,i}} + h_{p_{s,i}},$$
(6)

where $h_{v_{s,i}}$, $h_{p_{s,i}}$ are feature embeddings and position embeddings for i -th node in session s .

Additionally, the model incorporates a weighted graph attention layer (WGAT) to capture the entire neighborhood information of nodes. The model outputs a graph-level representation vector, denoted as:

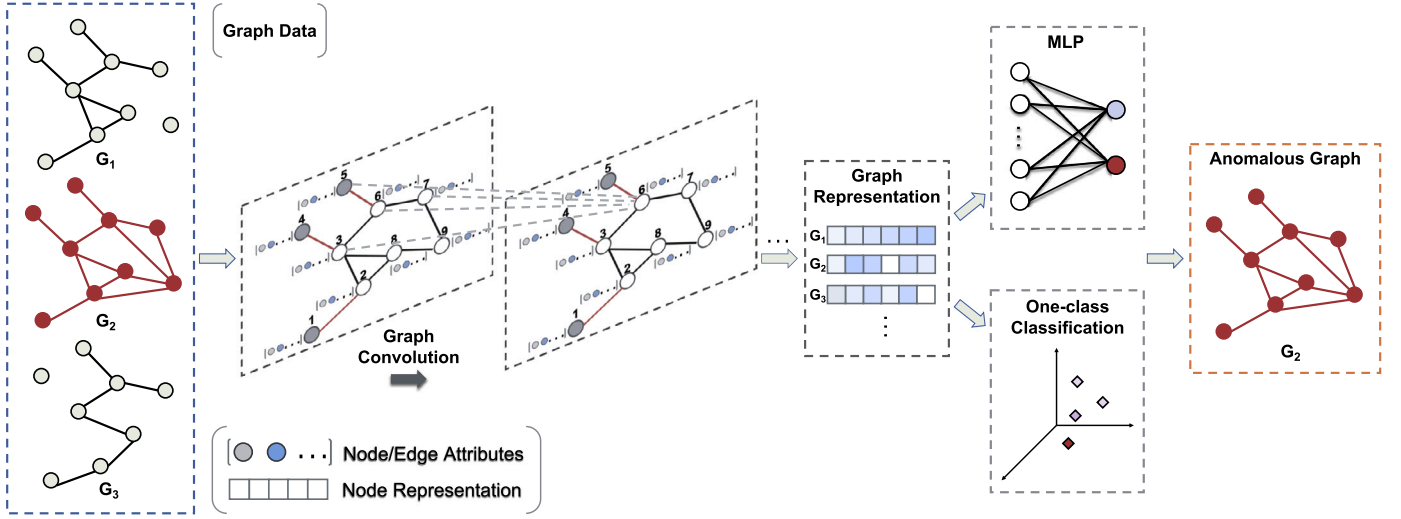


Fig. 5. A toy example of graph convolution for graph-level intrusion detection (static graph). Node features aggregation in graph convolution layers. A global graph embedding is derived through pooling layer. By generating representation for each single graph, anomaly graphs can be detected by MLP or One-class classification.

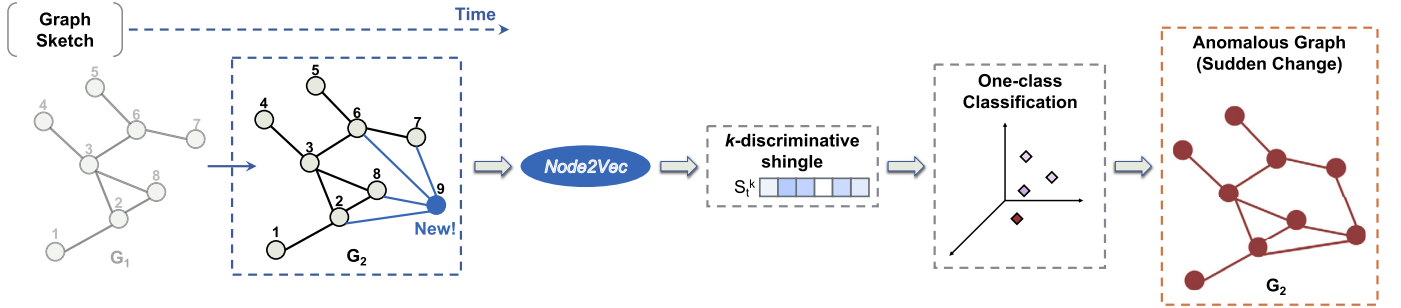


Fig. 6. A toy example of graph convolution for graph-level intrusion detection (dynamic graph). For graph sketch, the Node2Vec computes its k -discriminative shingle S^k . By generating representation for a graph sketch, anomalous graphs can be detected using one-class classification; a graph is deemed anomalous if it exhibits a sudden change.

$$h_{G_s} = \tanh \left(\sum_{i=1}^{|V_s|} \text{softmax} \left(f \left(h_{s,i}^L \right) \right) \odot \tanh \left(g \left(h_{s,i}^{LL} \right) \right) \right). \quad (7)$$

The CAN bus intrusion detection method proposed by Zhang et al. (2023a) is based on graph-level algorithms. This method examines the CAN message graph within a certain time interval to determine whether the network environment has been attacked during this period. To address the issue of highly imbalanced training data, a two-tier classifier cascade was developed, consisting of a binary classifier and a multi-class classifier. The system first detects attacks from the constructed graph, and once an attack is detected in the first-tier classifier, the sample is sent to the second-tier classifier for attack type classification. The model employs DGCNN (Zhang et al., 2018) to learn the graph structure. The graph convolution process is as follows:

$$w = f \left(\bar{D}^{-1} \bar{A} X W \right). \quad (8)$$

To adapt it for CAN message graphs, the authors made modifications to the original network. The input feature matrix simultaneously includes both node and edge matrices, represented as $X_{n \times (n+n')}$. Since CAN message graphs allow self-loops, $\bar{A} = A + I$, where the addition of the identity matrix I supports self-loops.

5.3. Graph-to-feature in host intrusion detection

Compared to network intrusion detection, host intrusion detection system (HIDS) offers low-level and fine-grained information (including file systems, system calls, and network events) that serve as the founda-

tion for detecting attacks that most network intrusion detection systems are unable to handle (Valeur et al., 2004). Effective graph representation is also crucial for building models capable of detecting complex attacks occurring at the host level. In this section, we will categorize our investigation into Provenance Graph and System Call Graph.

5.3.1. Provenance graph

Provenance Graphs serve as a promising data source for host-based intrusion detection systems due to their ability to trace the origin and transformation of data (Hassan et al., 2020; Zipperle et al., 2022). These directed acyclic graphs (DAGs) visually or structurally represent relationships between data items, processes, and agents involved in data creation, modification, or usage. Understanding the history, lineage, and processing of data is vital for intrusion detection.

However, analyzing the provenance graph is challenging due to its large scale and the presence of redundant nodes. For instance, DeepHunter (Wei et al., 2021) extracts indicators of compromise (IoCs) from provenance graphs, incorporating them into node attributes to reduce redundancy. It utilizes GCN to generate node-level embeddings for the query graph and the provenance graphs, followed by employing the global context-aware attention strategy from SimGNN (Bai et al., 2019) to obtain graph-level embeddings for both graphs. Based on the embeddings h_{G_q} and h_{G_p} of the two graphs, DeepHunter utilizes a relational learning network (such as NTN (Socher et al., 2013)) to learn the relationship between the two graphs and outputs a graph matching score s . The model aims to minimize the difference between the actual matching score and the output matching score.

THREATTRACE (Wang et al., 2022) offers a multi-model framework for real-time host monitoring, utilizing a novel feature extraction and label assignment method for semi-supervised intrusion detection. The framework learns different features of benign data in a supervised mode, training only on subgraphs to ensure scalability. In a streaming setting, it maintains a dynamic queue Q of abnormal nodes, issuing alerts when the number of anomalies exceeds a predefined threshold $\hat{T}$, thereby providing a research point for reducing false positives in streaming settings. Accurately and timely detecting intrusions from massive multi-source logs presents a challenge for modern systems.

OC-DHetGNN (Huang et al., 2022) initially modeled the provenance graph as a heterogeneous graph with attributes. This method allowed for the representation of various types of nodes and edges in the graph, thereby capturing the complex interactions among different system entities such as processes and files. Then, OC-DHetGNN employed a HetGNN (Zhang et al., 2019) module to learn the embeddings of the heterogeneous graph and its nodes. The embeddings of the heterogeneous graph and the nodes were respectively input into two one-class neural network modules to output anomaly scores. Paradise (Wu et al., 2023e) introduces a new extraction strategy, pruning and extracting process feature vectors from source dependencies at the system log level, and storing them in an efficient memory database. Paradise employs a multi-detector aggregation scheme, negotiating results from multiple detectors without additional communication overhead between detectors.

5.3.2. System call graph

System call graphs are integral for analyzing the behavior of applications, primarily focusing on the sequence and interaction of system calls during program execution. These graphs are essential for identifying abnormal or malicious activities within a system, providing insights into the runtime behavior of applications.

Lakha et al. (2022) mapped events and processes from system logs to nodes, constructing a graph structure with eventCode for events and processCode for processes serving as node identifiers. Connections between event nodes and their parent process nodes were established via the processCode, reflecting the relationship between events and processes. Thus, each node represented an individual event or process, while also depicting the dynamic interactions between them through edges. This provided a data structure for graph-based deep learning models that includes interaction information. After transforming logs into a graph structure, the NWR-GAE model (Tang et al., 2022) was employed to learn graph features, as this model is capable of representing the proximity and structure of nodes.

Xie et al. (2022) converted log sequences from system surveys into graph structures, where nodes represented log events, and edges between nodes denoted the spatial structural relationships among events. This transformation allowed the model to consider the relationships between events, rather than merely their counts or sequences. Xie et al. (2022) analyzed the constructed graphs using a customized graph transformer network, a model that integrated both the graph structure and node semantics.

5.4. GNN models deployment

The diverse methods of data collection and storage make the design and deployment of IDS challenging. Most IDS placement strategies are suboptimal, resulting in their inefficiency (Bhale et al., 2023). Proposing reasonable solutions for optimal IDS placement is a consideration in many research works.

There are two main challenges: the integration level between models and the computational efficiency of the models. Many recent methods combine RNN and GNN to capture both topological and temporal features of data. However, these methods either depend on the output of RNNs during the GNN graph embedding phase or simply integrate

GNN into the RNN architecture without accounting for their mutual interactions. This straightforward integration requires the models to be sequential, constraining scalability for extensive data. Another issue is that most of the memory consumption in existing architectures is spent on message passing within GNN, meaning subsequent simple models must wait for GNN computation to complete, leading to low efficiency.

GraphNET (Protogerou et al., 2021) presents a distributed deployment strategy for GNN models. This strategy involves deploying multi-agent systems across an entire internet of things (IoT) network, with each agent implementing a GNN model. To ensure connectivity throughout the IoT network, there is an exchange of information between each agent. GraphNET utilizes both edge multi-layer perceptron (MLP) and node MLP to extract data features.

Euler (King and Huang, 2023) permits several replicated GNNs to run independently on sequence encoders like RNNs, clearly enabling parallel execution of the GNN embedding process. Through this approach, GNNs encode network topologies at discrete moments, while RNNs capture dynamically changing network connections.

5.4.1. Deployment of GNNs in different domain

In this subsection, we further delve into the practical deployment of GNN models across various network domains, each presenting unique challenges and requiring tailored approaches.

The deployment of GNNs in IoT environments focuses on handling the vast network of interconnected devices and sensors. These networks are characterized by their heterogeneity and the need for real-time data processing. GNNs are particularly effective in modeling the complex interactions and communication patterns within IoT networks (Dong et al., 2023). For instance, Wu et al. (2023a) tackled the challenge of data scarcity in IoT intrusion detection (IID) by leveraging data from the network intrusion detection domain. It introduced a geometric graph alignment approach to overcome the geometric heterogeneities between these domains, facilitating better knowledge transfer for intrusion detection. The approach models each intrusion domain as a graph, with vertices and edges representing different intrusion categories.

In cyber-physical systems (CPS), GNNs play a crucial role in integrating physical processes with networked computing. These systems often form the backbone of critical infrastructure, where the detection of cyber-physical threats is paramount. Vincent et al. (2023) introduced a GCN framework to detect false data injection (FDI) attacks in smart grids. The GCN framework utilizes the graphical structures of the power network to analyze fluctuating state estimation values based on system topology. Liu et al. (2023b) proposed a method for generating synthetic topologies of cyber-physical power systems (CPS) using graph neural networks. Wu et al. (2023b) focused on industrial cyber-physical systems (ICPSs) and proposed a novel end-to-end physics-informed gated recurrent graph attention unit network for unsupervised anomaly detection. This method differs from conventional data-driven approaches by combining prior knowledge with process data. It features a physics-informed graph structure learning module to model dependencies among variables and introduces graph regularization based on underlying physics.

By examining these specific deployment scenarios, we underscore the importance of customizing GNN model deployment strategies to fit the unique requirements of different network domains. This not only enhances the effectiveness of the models in these settings but also provides insights into the practical considerations necessary for successful GNN deployment in real-world scenarios.

6. Open problems and future directions

In this section, we identify the open problems of existing GNN-based methods and outline promising research directions that are worth exploring in the future.

6.1. Enhanced data privacy measures in graph neural networks

The privacy concerns associated with GNNs handling sensitive graph data, including the challenges of data imbalance and missing values, necessitate the development of more sophisticated privacy-preserving mechanisms (Li et al., 2021a; Cai et al., 2021). Efforts to address these concerns should also consider the implications of the unavailability of labeled datasets, ensuring that the anonymization and encryption techniques do not further complicate these issues. Advanced techniques for anonymizing and encrypting graph data should ensure data integrity and utility for analysis purposes are maintained, while addressing these overarching challenges. This research could explore novel approaches, such as adapting differential privacy or homomorphic encryption, for graph-based data structures.

6.2. Advancements in model interpretability in network contexts

The complexity of GNNs poses challenges in interpretability, particularly crucial in security-sensitive environments (Keshk et al., 2023; García-Sigüenza et al., 2023). The interpretability issue is further compounded by high resource utilization in GNN workflows, making it essential to develop frameworks or tools that are not only interpretive but also resource-efficient. These should be capable of elucidating model decisions in an accessible manner, thereby fostering trust and informed decision-making among users and network administrators. The emphasis should be on creating user-friendly visualization tools and explainability algorithms to demystify GNN outputs in real-time.

6.3. Robustness and security in GNNs against adversarial attacks

The susceptibility of GNNs to adversarial attacks calls for enhanced security protocols and robust model architectures (Zhou et al., 2022). Future research directions should include designing GNNs with inherent resistance to adversarial inputs, possibly through the integration of adversarial training techniques. This involves improving the architecture and incorporating continuous monitoring and updating mechanisms to adapt to evolving adversarial strategies.

6.4. Optimization of graph data integration for host activity analysis

Efficiently integrating and analyzing data from provenance graphs and system call graphs remains a significant challenge. Future studies should aim to optimize graph representation and minimize redundancy in graph-to-feature transformation processes. This is particularly relevant in the context of host intrusion detection systems, where accurate and efficient processing of graph data is crucial for timely threat detection and response.

6.5. Development of heterogeneous graph neural networks

The need for effective analysis of heterogeneous graph data, particularly in network security, is becoming increasingly evident (Lee and Kim, 2022). Research should aim to develop GNN architectures capable of processing data simultaneously from various sources, such as workstations, servers, NIDS, HIDS, and firewalls. This would provide a comprehensive view of network security, enhancing the detection and mitigation of threats across diverse network components.

6.6. Dynamic graph algorithms for real-time network traffic analysis

The dynamic nature of network traffic necessitates the development of streaming graph algorithms for flow graph analysis. This emerging field holds the potential to transform network security by offering nuanced insights into data flow dynamics. Research should concentrate on creating algorithms capable of real-time analysis of streaming network data, enabling more proactive and effective network threat detection and prevention strategies.

7. Conclusion

The integration of graph neural networks into intrusion detection systems has marked a significant stride in the realm of network security. Their inherent ability to process and analyze graph data offers a unique advantage, especially when dealing with complex network structures and patterns. However, as with any emerging technology, graph neural networks have sparked much research and also face challenges in this domain. We systematically reviewed the fundamental concepts of intrusion detection and graph neural networks, then delved deeply into various aspects of applying graph neural networks to intrusion detection, including graph construction, network design, and model deployment methods. The insights and findings presented in this review provide a foundational framework and exhibition of work for future research. We also discussed open issues and future research directions in this field, including ensuring user data privacy and enhancing model interpretability and robustness, hoping to provide valuable references for scholars and engineers. Focusing on these challenges will not only ensure the effectiveness of graph neural networks but also adapt to the ever-changing landscape of network threats and security needs. The future of intrusion detection, supported by the capabilities of graph neural networks, looks promising. Continued research and innovation in this field will undoubtedly pave the way for a more secure and resilient network environment.

CRedit authorship contribution statement

Meihui Zhong: Writing – original draft, Software, Methodology, Investigation, Data curation. **Mingwei Lin:** Writing – review & editing, Validation, Supervision, Funding acquisition, Conceptualization. **Chao Zhang:** Writing – review & editing, Supervision. **Zeshui Xu:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This research work was supported by National Natural Science Foundation of China under Grant No. 62272103, Natural Science Foundation of Fujian Province of China under Grant Nos. 2022J06020 and 2023J02014, Young Top Talent of Young Eagle Program of Fujian Province, China under Grant No. F21E0011202B01.

References

- Abdel Wahab, O., 2022. Intrusion detection in the IoT under data and concept drifts: online deep learning approach. *IEEE Int. Things J.* 9, 19706–19716. <https://doi.org/10.1109/JIOT.2022.3167005>.
- Agillo, A., Bardhi, E., Conti, M., Lazzaretto, R., Losiouk, E., Omicini, A., 2023. Gnn4ifa: interest flooding attack detection with graph neural networks. In: 2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P), pp. 615–630.
- Alshamrani, A., Myneni, S., Chowdhary, A., Huang, D., 2019. A survey on advanced persistent threats: techniques, solutions, challenges, and research opportunities. *IEEE Commun. Surv. Tutor.* 21, 1851–1877. <https://doi.org/10.1109/COMST.2019.2891891>.
- Altat, T., Wang, X., Ni, W., Liu, R.P., Braun, R., 2023a. Ne-gconv: a lightweight node edge graph convolutional network for intrusion detection. *Comput. Secur.* 130, 103285. <https://doi.org/10.1016/j.cose.2023.103285>.
- Altat, T., Wang, X., Ni, W., Yu, G., Liu, R.P., Braun, R., 2023b. A new concatenated multi-graph neural network for IoT intrusion detection. *Int. Things J.* 22, 100818. <https://doi.org/10.1016/j.iot.2023.100818>.

- Ao, S., He, Y., Luo, N., Wang, X., Jiang, Z., Jiang, J., 2021. Modeling attackers based on heterogeneous graph through malicious http requests. In: 2021 IEEE 24th International Conference on Computer Supported Cooperative Work in Design (CSCWD), pp. 391–396.
- Bai, Y., Ding, H., Bian, S., Chen, T., Sun, Y., Wang, W., 2019. Simgnn: a neural network approach to fast graph similarity computation. In: Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining, pp. 384–392.
- Basati, A., Faghih, M.M., 2022. Pdae: efficient network intrusion detection in iot using parallel deep auto-encoders. *Inf. Sci.* 598, 57–74. <https://doi.org/10.1016/j.ins.2022.03.065>.
- Bhale, P., Chowdhury, D.R., Biswas, S., Nandi, S., 2023. Optimist: lightweight and transparent ids with optimum placement strategy to mitigate mixed-rate ddos attacks in iot networks. *IEEE Int. Things J.* 10, 8357–8370. <https://doi.org/10.1109/JIOT.2023.3234530>.
- Bilot, T., Madhoun, N.E., Agha, K.A., Zouaoui, A., 2023. Graph neural networks for intrusion detection: a survey. *IEEE Access* 11, 49114–49139. <https://doi.org/10.1109/ACCESS.2023.3275789>.
- Biswas, P., Samanta, T., Sanyal, J., 2023. Intrusion detection using graph neural network and Lyapunov optimization in wireless sensor network. *Multimed. Tools Appl.* 82, 14123–14134. <https://doi.org/10.1007/s11042-022-13992-9>.
- Bowman, B., Laprade, C., Ji, Y., Huang, H.H., 2020. Detecting lateral movement in enterprise computer networks with unsupervised graph AI. In: 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020), pp. 257–268. <https://www.usenix.org/conference/raid2020/presentation/bowman>.
- Buczak, A.L., Guven, E., 2016. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Commun. Surv. Tutor.* 18, 1153–1176. <https://doi.org/10.1109/COMST.2015.2494502>.
- Cai, H., Zheng, V.W., Chang, K.C.C., 2018. A comprehensive survey of graph embedding: problems, techniques, and applications. *IEEE Trans. Knowl. Data Eng.* 30, 1616–1637. <https://doi.org/10.1109/TKDE.2018.2807452>.
- Cai, Z., Xiong, Z., Xu, H., Wang, P., Li, W., Pan, Y., 2021. Generative adversarial networks: a survey toward private and secure applications. *ACM Comput. Surv.* 54. <https://doi.org/10.1145/3459992>.
- Caville, E., Lo, W.W., Layeghy, S., Portmann, M., 2022. Anomal-e: a self-supervised network intrusion detection system based on graph neural networks. *Knowl.-Based Syst.* 258, 110030. <https://doi.org/10.1016/j.knsys.2022.110030>.
- Chang, L., Branco, P., 2021. Graph-based solutions with residuals for intrusion detection: the modified e-graphsage and e-resgat algorithms. *arXiv:2111.13597*.
- Chen, M., Wei, Z., Huang, Z., Ding, B., Li, Y., 2020. Simple and deep graph convolutional networks. In: Proceedings of the 37th International Conference on Machine Learning, pp. 1725–1735. <https://proceedings.mlr.press/v119/chen20v.html>.
- Chen, Y., Wu, L., 2022. Graph Neural Networks: Graph Structure Learning. Springer, pp. 297–321. Chapter 14.
- Cheng, Q., Wu, C., Zhou, S., 2021. Discovering attack scenarios via intrusion alert correlation using graph convolutional networks. *IEEE Commun. Lett.* 25, 1564–1567. <https://doi.org/10.1109/LCOMM.2020.3048995>.
- Corona, I., Giacinto, G., Roli, F., 2013. Adversarial attacks against intrusion detection systems: taxonomy, solutions and open issues. *Inf. Sci.* 239, 201–225. <https://doi.org/10.1016/j.ins.2013.03.022>.
- Creech, G., Hu, J., 2014. A semantic approach to host-based intrusion detection systems using contiguous and discontinuous system call patterns. *IEEE Trans. Comput.* 63, 807–819. <https://doi.org/10.1109/TC.2013.13>.
- Cui, P., Wang, X., Pei, J., Zhu, W., 2019. A survey on network embedding. *IEEE Trans. Knowl. Data Eng.* 31, 833–852. <https://doi.org/10.1109/TKDE.2018.2849727>.
- Debar, H., Dacier, M., Wespi, A., 1999. Towards a taxonomy of intrusion-detection systems. *Comput. Netw.* 31, 805–822. [https://doi.org/10.1016/S1389-1286\(98\)00017-6](https://doi.org/10.1016/S1389-1286(98)00017-6).
- Deng, X., Zhu, J., Pei, X., Zhang, L., Ling, Z., Xue, K., 2023. Flow topology-based graph convolutional network for intrusion detection in label-limited iot networks. *IEEE Trans. Netw. Serv. Manag.* 20, 684–696. <https://doi.org/10.1109/TNSM.2022.3213807>.
- Dong, G., Tang, M., Wang, Z., Gao, J., Guo, S., Cai, L., Gutierrez, R., Campbell, B., Barnes, L.E., Boukhechba, M., 2023. Graph neural networks in iot: a survey. *ACM Trans. Sens. Netw.* 19. <https://doi.org/10.1145/3565973>.
- Du, J., Zhang, S., Wu, G., Moura, J.M.F., Kar, S., 2017. Topology adaptive graph convolutional networks. *CoRR*, arXiv:1710.10370.
- Duan, G., Lv, H., Wang, H., Feng, G., 2023. Application of a dynamic line graph neural network for intrusion detection with semisupervised learning. *IEEE Trans. Inf. Forensics Secur.* 18, 699–714. <https://doi.org/10.1109/TIFS.2022.3228493>.
- Fraihat, S., Makhadmeh, S., Awad, M., Al-Betar, M.A., Al-Redhaei, A., 2023. Intrusion detection system for large-scale iot network networks using machine learning with modified arithmetic optimization algorithm. *Int. Things* 22, 100819. <https://doi.org/10.1016/j.iot.2023.100819>.
- Frijji, H., Olivereau, A., Sarkiss, M., 2023. Efficient network representation for gnn-based intrusion detection. In: *Applied Cryptography and Network Security*, pp. 532–554.
- Gao, C., Zheng, Y., Li, N., Li, Y., Qin, Y., Piao, J., Quan, Y., Chang, J., Jin, D., He, X., Li, Y., 2023a. A survey of graph neural networks for recommender systems: challenges, methods, and directions. *ACM Trans. Recomm. Syst.* 1. <https://doi.org/10.1145/3568022>.
- Gao, M., Wu, L., Li, Q., Chen, W., 2023b. Anomaly traffic detection in iot security using graph neural networks. *J. Inf. Secur. Appl.* 76, 103532. <https://doi.org/10.1016/j.jisa.2023.103532>.
- García-Sigüenza, J., Llorens-Largo, F., Tortosa, L., Vicent, J.F., 2023. Explainability techniques applied to road traffic forecasting using graph neural network models. *Inf. Sci.* 645, 119320. <https://doi.org/10.1016/j.ins.2023.119320>.
- Gavel, S., Raghuvanshi, A.S., Tiwari, S., 2022. Maximum correlation based mutual information scheme for intrusion detection in the data networks. *Expert Syst. Appl.* 189, 116089. <https://doi.org/10.1016/j.eswa.2021.116089>.
- Gibert, D., Mateu, C., Planes, J., 2020. The rise of machine learning for detection and classification of malware: research developments, trends and challenges. *J. Netw. Comput. Appl.* 153, 102526. <https://doi.org/10.1016/j.jnca.2019.102526>.
- Gong, L., Cheng, Q., 2019. Exploiting edge features for graph neural networks. In: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9203–9211.
- Govindaraju, S., Vinisha, W.V.R., Shajin, F.H., Sivasakthi, D.A., 2022. Intrusion detection framework using auto-metric graph neural network optimized with hybrid woodpecker mating and capuchin search optimization algorithm in iot network. *Concurr. Comput., Pract. Exp.* 34, e7197. <https://doi.org/10.1002/cpe.7197>.
- Grover, A., Leskovec, J., 2016. Node2vec: scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 855–864.
- Guo, W., Qiu, H., Liu, Z., Zhu, J., Wang, Q., 2022. Gld-net: deep learning to detect ddos attack via topological and traffic feature fusion. *Comput. Intell. Neurosci.* 2022, 4611331. <https://doi.org/10.1155/2022/4611331>.
- Guo, Y., Peng, Y., Hao, R., Tang, X., 2023. Capturing spatial-temporal correlations with attention based graph convolutional network for network traffic prediction. *J. Netw. Comput. Appl.* 220, 103746. <https://doi.org/10.1016/j.jnca.2023.103746>.
- Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. In: *Advances in Neural Information Processing Systems*, pp. 1025–1035.
- Han, M., Feng, S., Chen, C.L.P., Xu, M., Qiu, T., 2019. Structured manifold broad learning system: a manifold perspective for large-scale chaotic time series analysis and prediction. *IEEE Trans. Knowl. Data Eng.* 31, 1809–1821. <https://doi.org/10.1109/TKDE.2018.2866149>.
- Hassan, W.U., Bates, A., Marino, D., 2020. Tactical provenance analysis for endpoint detection and response systems. In: 2020 IEEE Symposium on Security and Privacy (SP), pp. 1172–1189.
- Hu, B., Kamiya, K., Takahashi, K., Nakao, A., 2021. Multi-hop graph embedding for botnet detection. In: 2021 IEEE Global Communications Conference (GLOBECOM), pp. 1–6.
- Hu, X., Gao, W., Cheng, G., Li, R., Zhou, Y., Wu, H., 2023. Toward early and accurate network intrusion detection using graph embedding. *IEEE Trans. Inf. Forensics Secur.* 18, 5817–5831. <https://doi.org/10.1109/TIFS.2023.3318960>.
- Huang, H., Guo, S., Li, C., Sheng, J., Wang, L., Li, J., Liu, J., Zhong, S., 2021. Two-tier graph contextual embedding for cross-device user matching. In: Proceedings of the 30th ACM International Conference on Information & Knowledge Management, pp. 730–739.
- Huang, Z., Gu, Y., Zhao, Q., 2022. One-class directed heterogeneous graph neural network for intrusion detection. In: Proceedings of the 2022 6th International Conference on Innovation in Artificial Intelligence. Association for Computing Machinery, New York, NY, USA, pp. 178–184.
- Hwang, K., Cai, M., Chen, Y., Qin, M., 2007. Hybrid intrusion detection with weighted signature generation over anomalous Internet episodes. *IEEE Trans. Dependable Secure Comput.* 4, 41–55. <https://doi.org/10.1109/TDSC.2007.9>.
- Islam, R., Refat, R.U.D., Yerram, S.M., Malik, H., 2022. Graph-based intrusion detection system for controller area networks. *IEEE Trans. Intell. Transp. Syst.* 23, 1727–1736. <https://doi.org/10.1109/TITS.2020.3025685>.
- Jain, M., Kaur, G., Saxena, V., 2022. A k-means clustering and svm based hybrid concept drift detection technique for network anomaly detection. *Expert Syst. Appl.* 193, 116510. <https://doi.org/10.1016/j.eswa.2022.116510>.
- Jamalipour, A., Murali, S., 2022. A taxonomy of machine-learning-based intrusion detection systems for the Internet of things: a survey. *IEEE Int. Things J.* 9, 9444–9466. <https://doi.org/10.1109/JIOT.2021.3126811>.
- Javed, Y., Khayat, M.A., Elghariani, A.A., Ghafoor, A., 2023. Prism: a hierarchical intrusion detection architecture for large-scale cyber networks. *IEEE Trans. Dependable Secure Comput.* 20, 5070–5086. <https://doi.org/10.1109/TDSC.2023.3240315>.
- Jedh, M., Ben Othmane, L., Ahmed, N., Bhargava, B., 2021. Detection of message injection attacks onto the can bus using similarities of successive messages-sequence graphs. *IEEE Trans. Inf. Forensics Secur.* 16, 4133–4146. <https://doi.org/10.1109/TIFS.2021.3098162>.
- Kaur, R., Gabrijelčić, D., Klobučar, T., 2023. Artificial intelligence for cybersecurity: literature review and future research directions. *Inf. Fusion* 97, 101804. <https://doi.org/10.1016/j.inffus.2023.101804>.
- Keshk, M., Koroniotis, N., Pham, N., Moustafa, N., Turnbull, B., Zomaya, A.Y., 2023. An explainable deep learning-enabled intrusion detection framework in iot networks. *Inf. Sci.* 639, 119000. <https://doi.org/10.1016/j.ins.2023.119000>.
- King, I.J., Huang, H.H., 2023. Euler: detecting network lateral movement via scalable temporal link prediction. *ACM Trans. Priv. Secur.* 26. <https://doi.org/10.1145/3588771>.
- Kipf, T.N., Welling, M., 2017. Semi-supervised classification with graph convolutional networks. In: *International Conference on Learning Representations*, pp. 1–14. <https://openreview.net/forum?id=SJU4ayYgl>.

- Ko, H., Praca, I., Choi, S.G., 2023. Anomaly detection analysis based on correlation of features in graph neural network. *Multimed. Tools Appl.* <https://doi.org/10.1007/s11042-023-15635-z>.
- Lakha, B., Mount, S.L., Serra, E., Cuzzocrea, A., 2022. Anomaly detection in cybersecurity events through graph neural network and transformer based model: a case study with beth dataset. In: *2022 IEEE International Conference on Big Data (Big Data)*, pp. 5756–5764.
- Lan, J., Lu, J.Z., Wan, G.G., Wang, Y.Y., Huang, C.Y., Zhang, S.B., Huang, Y.Y., Ma, J.N., Damaševičius, R., 2022. E-minbatch graphsage: an industrial Internet attack detection model. *Secur. Commun. Netw.* 2022, 5363764. <https://doi.org/10.1155/2022/5363764>.
- Landauer, M., Skopik, F., Frank, M., Hotwagner, W., Wurzenberger, M., Rauber, A., 2023. Maintainable log datasets for evaluation of intrusion detection systems. *IEEE Trans. Dependable Secure Comput.* 20, 3466–3482. <https://doi.org/10.1109/TDSC.2022.3201582>.
- Lazzarini, R., Tianfield, H., Charissis, V., 2023. A stacking ensemble of deep learning models for iot intrusion detection. *Knowl.-Based Syst.* 279, 110941. <https://doi.org/10.1016/j.knsys.2023.110941>.
- Lee, M., Kim, S.B., 2022. Hapgnn: hop-wise attentive pagerank-based graph neural network. *Inf. Sci.* 613, 435–452. <https://doi.org/10.1016/j.ins.2022.09.041>.
- Li, K., Luo, G., Ye, Y., Li, W., Ji, S., Cai, Z., 2021a. Adversarial privacy-preserving graph embedding against inference attack. *IEEE Int. Things J.* 8, 6904–6915. <https://doi.org/10.1109/JIOT.2020.3036583>.
- Li, X., Zhu, M., Yang, L.T., Xu, M., Ma, Z., Zhong, C., Li, H., Xiang, Y., 2021b. Sustainable ensemble learning driving intrusion detection model. *IEEE Trans. Dependable Secure Comput.* 18, 1591–1604. <https://doi.org/10.1109/TDSC.2021.3066202>.
- Liu, M., Xue, Z., Xu, X., Zhong, C., Chen, J., 2018. Host-based intrusion detection system with system calls: review and future trends. *ACM Comput. Surv.* 51. <https://doi.org/10.1145/3214304>.
- Liu, Y., Ding, K., Wang, J., Lee, V., Liu, H., Pan, S., 2023a. Learning strong graph neural networks with weak information. In: *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 1559–1571.
- Liu, Y., Xie, H., Presekal, A., Stefanov, A., Palensky, P., 2023b. A gnn-based generative model for generating synthetic cyber-physical power system topology. *IEEE Trans. Smart Grid* 14, 4968–4971. <https://doi.org/10.1109/TSG.2023.3304134>.
- Lo, W.W., Layeghy, S., Sarhan, M., Gallagher, M., Portmann, M., 2022. E-graphsage: a graph neural network based intrusion detection system for iot. In: *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, pp. 1–9.
- Louk, M.H.L., Tama, B.A., 2023. Dual-ids: a bagging-based gradient boosting decision tree model for network anomaly intrusion detection system. *Expert Syst. Appl.* 213, 119030. <https://doi.org/10.1016/j.eswa.2022.119030>.
- Ma, X., Wu, J., Xue, S., Yang, J., Zhou, C., Sheng, Q.Z., Xiong, H., Akoglu, L., 2023. A comprehensive survey on graph anomaly detection with deep learning. *IEEE Trans. Knowl. Data Eng.* 35, 12012–12038. <https://doi.org/10.1109/TKDE.2021.3118815>.
- Messai, M.L., Seba, H., 2023. Iot network attack detection: leveraging graph learning for enhanced security. In: *Proceedings of the 18th International Conference on Availability, Reliability and Security*, pp. 1–7.
- Nguyen, H., Kashef, R., 2023. Ts-ids: traffic-aware self-supervised learning for iot network intrusion detection. *Knowl.-Based Syst.* 279, 110966. <https://doi.org/10.1016/j.knsys.2023.110966>.
- Nguyen, T.T., Reddi, V.J., 2023. Deep reinforcement learning for cyber security. *IEEE Trans. Neural Netw. Learn. Syst.* 34, 3779–3795. <https://doi.org/10.1109/TNNLS.2021.3121870>.
- Paudel, R., Eberle, W., 2020. Snapsketch: graph representation approach for intrusion detection in a streaming graph. In: *Proceedings of the 16th International Workshop on Mining and Learning with Graphs (MLG)*, p. null.
- Pontarelli, S., Bianchi, G., Teofilii, S., 2013. Traffic-aware design of a high-speed fpga network intrusion detection system. *IEEE Trans. Comput.* 62, 2322–2334. <https://doi.org/10.1109/TC.2012.105>.
- Pouyanfar, S., Sadiq, S., Yan, Y., Tian, H., Tao, Y., Reyes, M.P., Shyu, M.L., Chen, S.C., Iyengar, S.S., 2018. A survey on deep learning: algorithms, techniques, and applications. *ACM Comput. Surv.* 51. <https://doi.org/10.1145/3234150>.
- Protogerou, A., Papadopoulos, S., Drosou, A., Tzovaras, D., Refanidis, I., 2021. A graph neural network method for distributed anomaly detection in iot. *Evolv. Syst.* 12, 19–36. <https://doi.org/10.1007/s12530-020-09347-0>.
- Pujol-Perich, D., Suarez-Varela, J., Cabellos-Aparicio, A., Barlet-Ros, P., 2022. Unveiling the potential of graph neural networks for robust intrusion detection. *ACM SIGMETRICS Perform. Eval. Rev.* 49, 111–117. <https://doi.org/10.1145/3543146.3543171>.
- Qi, G.J., Luo, J., 2022. Small data challenges in big data era: a survey of recent progress on unsupervised and semi-supervised methods. *IEEE Trans. Pattern Anal. Mach. Intell.* 44, 2168–2187. <https://doi.org/10.1109/TPAMI.2020.3031898>.
- Qian, C., Yu, W., Lu, C., Griffith, D., Gollmie, N., 2022. Toward generative adversarial networks for the industrial Internet of things. *IEEE Int. Things J.* 9, 19147–19159. <https://doi.org/10.1109/JIOT.2022.3163894>.
- Reka, R., Karthick, R., Saravana Ram, R., Singh, G., 2024. Multi head self-attention gated graph convolutional network based multi-attack intrusion detection in manet. *Comput. Secur.* 136, 103526. <https://doi.org/10.1016/j.cose.2023.103526>.
- Ren, H., Tang, Y., Dong, W., Ren, S., Jiang, L., 2023. Duen: dynamic ensemble handling class imbalance in network intrusion detection. *Expert Syst. Appl.* 229, 120420. <https://doi.org/10.1016/j.eswa.2023.120420>.
- Singh, A., Amutha, J., Nagar, J., Sharma, S., 2023. A deep learning approach to predict the number of k-barriers for intrusion detection over a circular region using wireless sensor networks. *Expert Syst. Appl.* 211, 118588. <https://doi.org/10.1016/j.eswa.2022.118588>.
- Socher, R., Chen, D., Manning, C.D., Ng, A.Y., 2013. Reasoning with neural tensor networks for knowledge base completion. In: *Proceedings of the 26th International Conference on Neural Information Processing Systems*, vol. 1, pp. 926–934. https://proceedings.neurips.cc/paper_files/paper/2013/file/b337e84de8752b27eda3a12363109e80-Paper.pdf.
- Sun, X., Yang, J., 2022. Hetglm: lateral movement detection by discovering anomalous links with heterogeneous graph neural network. In: *2022 IEEE International Performance, Computing, and Communications Conference (IPCCC)*, pp. 404–411.
- Sun, Y., Zhu, D., Du, H., Tian, Z., 2023. Mhmf: multi-hop heterogeneous neighborhood information fusion graph representation learning. *IEEE Trans. Knowl. Data Eng.* 35, 7192–7205. <https://doi.org/10.1109/TKDE.2022.3186158>.
- Tang, M., Li, P., Yang, C., 2022. Graph auto-encoder via neighborhood Wasserstein reconstruction. In: *International Conference on Learning Representations*, pp. 1–17. <https://openreview.net/forum?id=ATUh28lnSuW>.
- Umer, M.F., Sher, M., Bi, Y., 2017. Flow-based intrusion detection: techniques and challenges. *Comput. Secur.* 70, 238–254. <https://doi.org/10.1016/j.cose.2017.05.009>.
- Valeur, F., Vigna, G., Kruegel, C., Kemmerer, R., 2004. Comprehensive approach to intrusion detection alert correlation. *IEEE Trans. Dependable Secure Comput.* 1, 146–169. <https://doi.org/10.1109/TDSC.2004.21>.
- Vincent, E., Korki, M., Seyedmahmoudian, M., Stojcevski, A., Mekhilef, S., 2023. Detection of false data injection attacks in cyber-physical systems using graph convolutional network. *Electr. Power Syst. Res.* 217, 109118. <https://doi.org/10.1016/j.epr.2023.109118>.
- Wan, Y., Liu, Y., Wang, D., Wen, Y., 2021. Glad-paw: graph-based log anomaly detection by position aware weighted graph attention network. In: *Advances in Knowledge Discovery and Data Mining*, pp. 66–77.
- Wang, S., Wang, Z., Zhou, T., Sun, H., Yin, X., Han, D., Zhang, H., Shi, X., Yang, J., 2022. Threatrace: detecting and tracing host-based threats in node level through provenance graph learning. *IEEE Trans. Inf. Forensics Secur.* 17, 3972–3987. <https://doi.org/10.1109/TIFS.2022.3208815>.
- Wang, W., Shang, Y., He, Y., Li, Y., Liu, J., 2020. Botmark: automated botnet detection with hybrid analysis of flow-based and graph-based traffic behaviors. *Inf. Sci.* 511, 284–296. <https://doi.org/10.1016/j.ins.2019.09.024>.
- Wang, X., Wang, X., He, M., Zhang, M., Lu, Z., 2024. Spatial-temporal graph model based on attention mechanism for anomalous iot intrusion detection. *IEEE Trans. Ind. Inform.* 20 (3), 3497–3509. <https://doi.org/10.1109/TII.2023.3308784>.
- Wang, Y., Li, J., Zhao, W., Han, Z., Zhao, H., Wang, L., He, X., 2023. N-stgar: spatio-temporal graph neural network based network intrusion detection for near-Earth remote sensing. *Remote Sens.* 15. <https://doi.org/10.3390/rs15143611>.
- Wei, R., Cai, L., Zhao, L., Yu, A., Meng, D., 2021. Deephunter: a graph neural network based approach for robust cyber threat hunting. In: *Security and Privacy in Communication Networks*, pp. 3–24.
- Weller-Fahy, D.J., Borghetti, B.J., Sodemann, A.A., 2015. A survey of distance and similarity measures used within network intrusion anomaly detection. *IEEE Commun. Surv. Tutor.* 17, 70–91. <https://doi.org/10.1109/COMST.2014.2336610>.
- Wenshu Xu, M.L., 2023. Information security evaluation of industrial control systems using probabilistic linguistic mcdm method. *Comput. Mater. Continua* 77, 199–222. <https://doi.org/10.32604/cmc.2023.041475>.
- Wu, J., Dai, H., Wang, Y., Ye, K., Xu, C., 2023a. Heterogeneous domain adaptation for iot intrusion detection: a geometric graph alignment approach. *IEEE Int. Things J.* 10, 10764–10777. <https://doi.org/10.1109/JIOT.2023.3239872>.
- Wu, W., Song, C., Zhao, J., Xu, Z., 2023b. Physics-informed gated recurrent graph attention unit network for anomaly detection in industrial cyber-physical systems. *Inf. Sci.* 629, 618–633. <https://doi.org/10.1016/j.ins.2023.01.136>.
- Wu, Y., Nie, L., Wang, S., Ning, Z., Li, S., 2023c. Intelligent intrusion detection for Internet of things security: a deep convolutional generative adversarial network-enabled approach. *IEEE Int. Things J.* 10, 3094–3106. <https://doi.org/10.1109/JIOT.2021.3112159>.
- Wu, Y., Nie, L., Xiong, X., Sadoun, B., Yang, L., Ning, Z., 2023d. Incremental update intrusion detection for industry 5.0 security: a graph attention network-enabled approach. *IEEE Trans. Consum. Electron.*, 1–1. <https://doi.org/10.1109/TCE.2023.3331907>.
- Wu, Y., Xie, Y., Liao, X., Zhou, P., Feng, D., Wu, L., Li, X., Wildani, A., Long, D., 2023e. Paradise: real-time, generalized, and distributed provenance-based intrusion detection. *IEEE Trans. Dependable Secure Comput.* 20, 1624–1640. <https://doi.org/10.1109/TDSC.2022.3160879>.
- Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Yu, P.S., 2021. A comprehensive survey on graph neural networks. *IEEE Trans. Neural Netw. Learn. Syst.* 32, 4–24. <https://doi.org/10.1109/TNNLS.2020.2978386>.
- Xiao, J., Yang, L., Zhong, F., Chen, H., Li, X., 2023. Robust anomaly-based intrusion detection system for in-vehicle network by graph neural network framework. *Appl. Intell.* 53, 3183–3206. <https://doi.org/10.1007/s10489-022-03412-8>.
- Xiao, T., Chen, Z., Wang, D., Wang, S., 2021. Learning how to propagate messages in graph neural networks. In: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pp. 1894–1903.
- Xie, Y., Zhang, H., Babar, M.A., 2022. Loggd: detecting anomalies from system logs with graph neural networks. In: *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*, pp. 299–310.

- Xu, K., Hu, W., Leskovec, J., Jegelka, S., 2019. How powerful are graph neural networks? In: International Conference on Learning Representations, p. nill. <https://openreview.net/forum?id=ryGs6iA5Km>.
- Yan, H., Li, X., Zhang, W., Wang, R., Li, H., Zhao, X., Li, F., Lin, X., 2024. Automatic evasion of machine learning-based network intrusion detection systems. *IEEE Trans. Dependable Secure Comput.* 21 (1), 153–167. <https://doi.org/10.1109/TDSC.2023.3247585>.
- Ye, N., Emran, S., Chen, Q., Vilbert, S., 2002. Multivariate statistical analysis of audit trails for host-based intrusion detection. *IEEE Trans. Comput.* 51, 810–820. <https://doi.org/10.1109/TC.2002.1017701>.
- Yehezkel, A., Elyashiv, E., 2022. A gnn-based approach for detecting network anomalies from small traffic samples. In: 2022 IEEE International Conference on Big Data (Big Data), pp. 6838–6840.
- Yu, L., Sun, L., Du, B., Liu, C., Lv, W., Xiong, H., 2023. Heterogeneous graph representation learning with relation awareness. *IEEE Trans. Knowl. Data Eng.* 35, 5935–5947. <https://doi.org/10.1109/TKDE.2022.3160208>.
- Yuan, H., Yu, H., Gui, S., Ji, S., 2023. Explainability in graph neural networks: a taxonomic survey. *IEEE Trans. Pattern Anal. Mach. Intell.* 45, 5782–5799. <https://doi.org/10.1109/TPAMI.2022.3204236>.
- Zerhoubi, S., Granitzer, M., Garchery, M., 2020. Improving intrusion detection systems using zero-shot recognition via graph embeddings. In: 2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC), pp. 790–797.
- Zhang, C., Song, D., Huang, C., Swami, A., Chawla, N.V., 2019. Heterogeneous graph neural network. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, Association for Computing Machinery, pp. 793–803.
- Zhang, G., Li, Z., Huang, J., Wu, J., Zhou, C., Yang, J., Gao, J., 2022a. Efraudcom: an e-commerce fraud detection system via competitive graph neural networks. *ACM Trans. Inf. Syst.* 40. <https://doi.org/10.1145/3474379>.
- Zhang, H., Zeng, K., Lin, S., 2023a. Federated graph neural network for fast anomaly detection in controller area networks. *IEEE Trans. Inf. Forensics Secur.* 18, 1566–1579. <https://doi.org/10.1109/TIFS.2023.3240291>.
- Zhang, M., Cui, Z., Neumann, M., Chen, Y., 2018. An end-to-end deep learning architecture for graph classification. In: Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence, pp. 4438–4445.
- Zhang, Y., Yang, C., Huang, K., Li, Y., 2023b. Intrusion detection of industrial Internet-of-things based on reconstructed graph neural networks. *IEEE Trans. Netw. Sci. Eng.* 10, 2894–2905. <https://doi.org/10.1109/TNSE.2022.3184975>.
- Zhang, Z., Cui, P., Zhu, W., 2022b. Deep learning on graphs: a survey. *IEEE Trans. Knowl. Data Eng.* 34, 249–270. <https://doi.org/10.1109/TKDE.2020.2981333>.
- Zheng, J., Li, D., 2019. Gcn-tc: combining trace graph with statistical features for network traffic classification. In: ICC 2019 - 2019 IEEE International Conference on Communications (ICC), pp. 1–6.
- Zheng, L., Li, Z., Li, J., Li, Z., Gao, J., 2019. Addgraph: anomaly detection in dynamic graph using attention-based temporal gcn. In: Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19, pp. 4419–4425.
- Zhong, M., Lin, M., He, Z., 2023. Dynamic multi-scale topological representation for enhancing network intrusion detection. *Comput. Secur.* 135, 103516. <https://doi.org/10.1016/j.cose.2023.103516>.
- Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., Sun, M., 2020. Graph neural networks: a review of methods and applications. *AI Open* 1, 57–81. <https://doi.org/10.1016/j.aiopen.2021.01.001>.
- Zhou, X., Liang, W., Li, W., Yan, K., Shimizu, S., Wang, K.I.K., 2022. Hierarchical adversarial attacks against graph-neural-network-based iot network intrusion detection system. *IEEE Int. Things J.* 9, 9310–9319. <https://doi.org/10.1109/JIOT.2021.3130434>.
- Zhu, H., Lu, J., 2022. Graph-based intrusion detection system using general behavior learning. In: GLOBECOM 2022 - 2022 IEEE Global Communications Conference, pp. 2621–2626.
- Zipperle, M., Gottwalt, F., Chang, E., Dillon, T., 2022. Provenance-based intrusion detection systems: a survey. *ACM Comput. Surv.* 55. <https://doi.org/10.1145/3539605>.